



SECRI033-03 COMPUTER ORGANIZATION AND ARCHITECTURE

SEMESTER 2, SESSION 2025/2026

PROJECT REPORT

LECTURER: DR. MD SAH HJ SALAM

GROUP MEMBERS:

			
CHENG ZHI MIN A25CS0050	LIM LI JING A25CS0248	LIM LI NING A25CS0249	NG XUAN YEE A25CS0291

VIDEO LINKS:

Tool 1 (NG XUAN YEE): [Tool 1 Cinebench 2026](#)

Tool 2 (LIM LI NING): [Tool 2 HWiNFO64](#)

Tool 3 (CHENG ZHI MIN): [Tool 3 PassMark PerformanceTest](#)

Tool 4 (LIM LI JING): [Tool 4 CrystalDiskMark](#)

TABLE OF CONTENTS

1.0 INTRODUCTION.....	3
2.0 TOOLS INSTALLATION AND CONFIGURATION.....	6
2.1 Details on Installation and Configuration of Tool 1.....	6
2.1.1 Tool Overview.....	6
2.1.2 Installation and Environment Setup.....	6
2.1.3 Brief Operational Manual for Execution.....	7
2.2 Details on Installation and Configuration of Tool 2.....	8
2.2.1 Tool Overview.....	8
2.2.2 Installation and Environment Setup.....	8
2.2.3 HWiNFO64 Sensor Configuration.....	10
2.2.4 Brief Operational Manual for Execution.....	11
2.3 Details on Installation and Configuration of Tool 3.....	12
2.3.1 Tool Overview.....	12
2.3.2 Installation and Environment Setup.....	12
2.3.3 Brief Operational Manual for Execution.....	14
2.4 Details on Installation and Configuration of Tool 4.....	15
2.4.1 Tool Overview.....	15
2.4.2 Installation and Environment Setup.....	16
2.4.3 Workload Parameter Configuration Matrix.....	17
2.4.4 Brief Operational Manual for Execution.....	18
3.0 PERFORMANCE BENCHMARK ANALYSIS AND DISCUSSIONS.....	19
3.1 Detailed Explanation on Workload A and Workload B.....	19
3.2 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload A.....	21
3.3 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload B.....	25
3.4 Analysis and Discussions.....	28
4.0 CONCLUSION AND REFLECTION.....	43
4.1 Conclusion.....	43
4.2 Reflection.....	44
5.0 REFERENCES.....	45
Appendix A: Task Distribution Table.....	45
Appendix B: Group Pictures.....	45
Appendix C: Artifact Deliverables.....	45

1.0 INTRODUCTION

Tool 1: Cinebench 2026 / R23 (Windows Edition)

An illustration of a cross-platform, synthetic microarchitectural benchmarking software application extensively used in conducting performance tests on personal computers is served by Cinebench 2026 / R23 (Windows Edition). Various types of speed testing methods may be performed using such software in order to help determine the performance rate of the tested computer and compare it with that of other computers. This explains why the particular software is considered very handy when measuring the performance rate of the computer. Taking into consideration all of this, the particular software will be very convenient in measuring the performance of CPUs in Workload A and B as per the COA Performance Measurement Architecture Matrix.

The version used in this project is Cinebench 2026 (Windows 64-bit Edition), which is the latest stable release as of 2026. As of the most recent builds, all standard rendering tests are now free with no license key required for evaluation. The software can be downloaded from the official Maxon website at <https://www.maxon.net/en/downloads/cinebench-downloads>.

Advantages

The use of the Cinebench 2026 / R23 application is particularly advantageous architecturally because of its ability to achieve a pure microarchitectural isolation. This is due to the fact that the test is designed specifically so that its load is directly dependent on CPU ALUs/FPU operations. As a result, such a test does not depend on the speed of reading/writing data from/into a computer's storage drives or memory latencies, thus providing raw results showing how fast the processor can calculate something. Additionally, vector computations put maximum pressure on the CPU, forcing it to operate under its highest possible electrical loads. Combined with the advanced test options window, this allows a user to tweak test parameters and get continuous testing either for 10 minutes or 30 minutes.

Disadvantages

However, Cinebench 2026 / R23's specialized approach also leads to some specific disadvantages on a microarchitectural level that should be taken into consideration. In particular, the nature of the test makes it a purely synthetic rendering benchmark that uses highly predictable floating-point operations, which do not emulate user branch prediction or common data processing such as database querying and sorting of files sequentially. The specialized nature of the workload results in an incomplete understanding of real-world computing processes. Furthermore, Cinebench 2026 / R23 does not account for the speed of data transmission within the computer system, as it completely isolates the internal execution pipeline of the processor from the rest of the memory hierarchy.

Tool 2: HWiNFO64 (Windows Edition)

Hardware Information for Windows (Windows Edition), abbreviated HWiNFO64, is an excellent computer program for real-time observation and data collection about the performance of the processor within a personal computer. This tool supports sensor monitoring capabilities through which a user can gather details about the CPU's temperature, clock speed, power usage, and thermal throttle state under diverse workload scenarios. This information comes handy when assessing the performance of a CPU, the efficiency of its pipeline, and thermal control in comparison of Workload A and Workload B using the COA Performance Measurement Architecture Matrix.

The version of this software used in this project is HWiNFO64 v8.34. As of June 2026, it is the most recent and stable version of the software currently in existence. The tool does not require any license

key for activation; it can be downloaded for free from the website <https://sourceforge.net/projects/hwinfo/>.

Advantages

A detailed control matrix over the poll intervals for hardware sensors and level of log detail is available through HWiNFO64, allowing the user to configure the Global Polling Period and Column Filtering using only a couple of mouse clicks. Hence, insights into the operations of thermal throttling and frequency downclocking under extended periods of multi-core load become available. CSV-formatted log files with time-stamped sensor values can be created, and further, the data can be exported to spreadsheet tools for quantitative analysis. Besides, as the metrics are represented using physical units (MHz for clock frequency, Watts for power consumption, and Yes/No for throttling state), the calculation of COA metrics such as Clock Frequency Delta (ΔT), Thermal Throttling Frequency, and Dynamic Power Consumption becomes a simple task, with all of these metrics calculated from the exported numbers. Moreover, HWiNFO64 allows core-by-core monitoring, providing individual percentages of core usage and core temperature information. These details, especially core usage statistics, provide insight into Windows' scheduler strategy for thread migration in single-core loads (Workload B), which includes moving the thread from one core to another to avoid the creation of a thermal hotspot.

Disadvantages

The HWiNFO64 score concentrates on the recording of sensor information and hardware monitoring, and problems may arise due to an inability to correlate the collected data with the performance scores obtained during benchmark testing (e.g., Cinebench points). If the polling rate is too high (e.g., >5000 ms), unreliable results will be obtained, since the data obtained from the sensors' buffer will be heavily smoothed out despite the presence of rapid temperature changes in the core of the processor. No instruction-level measures are offered by the monitoring utility (MIPS - Million Instructions Per Second, CPI - Cycles Per Instruction, etc.), which means that other software products must be used if a comprehensive analysis of performance is needed. Last but not least, continuous collection of sensor data and saving it to a CSV file results in enormous files (the file size of a 30-minute run can easily exceed 50MB).

Tool 3: PassMark PerformanceTest (Windows Edition)

PassMark PerformanceTest (Windows Edition) is a general-purpose benchmarking tool designed to evaluate and compare PC hardware performance through a standardized suite of objective speed tests. Its component-level scoring system makes it well-suited for measuring CPU performance across both Workload A and Workload B as defined in the COA Performance Measurement Architecture Matrix.

The version used in this project is PassMark PerformanceTest 11.1 Build 1010, which is the latest stable release as of 2026. As of the most recent builds, all standard tests are now free with no license key required. The software can be downloaded from the official PassMark website at <https://www.passmark.com/products/performancectest/download.php>.

Advantages

PassMark PerformanceTest includes an indexed global database that allows users to compare their results against machines from around the world, providing clear context for how a system performs relative to others. The application features a straightforward interface where all tests and options are accessible within a few clicks, and results can be exported as reports or graphs in multiple formats.

Since scores are reported in standard units such as MOps/Sec and GFLOPS, COA metrics like MIPS can be derived directly and consistently from the benchmark output.

Disadvantages

PassMark's combined scoring system can make direct comparisons with real-world performance difficult, and cross-version comparisons should be approached with caution as Version 11 yields notably lower 2D/3D scores than Version 10 on identical hardware. The tool also lacks microarchitectural detail such as cache miss rates and branch prediction data, requiring supplementary tools for a more thorough COA analysis. Additionally, network testing and advanced GPU calculations are locked behind a paid license.

Tool 4: CrystalDiskMark (Window Edition)

An illustration of a specialized benchmarking software application extensively used in conducting storage subsystem performance tests on personal computers is served by CrystalDiskMark (Windows Edition). Different kinds of objective data transfer speed tests can be carried out, by which the input/output operational capabilities and bus efficiencies of the assessed storage drive are assisted in being determined and compared under variable traffic conditions. It is due to this characteristic that the software is found to be extremely handy in measuring the performance rate of non-volatile solid-state drives (SSD) or hard disk drives (HDD). In light of this, this software can be conveniently used in measuring the performance of storage controllers and memory buses in Workload A and B according to the COA Performance Measurement Architecture Matrix.

The version used in this project is CrystalDiskMark 9.0.3 x64, which is the latest stable release as of 2026. As of the most recent builds, all standard testing profiles are open-source and free with no license key required. The software can be downloaded from the official CrystalMark website at <https://crystalmark.info/en/software/crystaldiskmark/>.

Advantages

An explicit control matrix over software and hardware execution abstraction layers is provided by CrystalDiskMark, enabling users to adjust the Queue Depth (Q) and Thread Count (T) within just a couple of mouse clicks. Consequently, an understanding of how native hardware command scheduling operates under stress can be easily gained. Reports and granular performance summaries related to the obtained results can be generated, and the data can be effortlessly exported or copied to the clipboard. Furthermore, because the results are presented in standard units (MB/s, IOPS), COA metrics like total storage bus throughput and transaction frequency can be easily calculated and evaluated based on these figures.

Disadvantages

CrystalDiskMark score is highly focused on pure I/O throughput data, difficulties can sometimes be encountered when direct tracking of internal CPU pipeline mechanics is attempted. Unreliable conclusions regarding sustained media speed limits may be reached if hardware-side volatile DRAM or SLC caching mechanisms are active, as substantially higher burst scores are yielded by the cache line fill buffers, even when the physical non-volatile storage array is experiencing heavy data degradation. No microarchitectural processor metrics, such as branch predictions or hardware interrupt trace logs, are provided by the benchmarking package, meaning that additional tools must be utilized if a more comprehensive assessment of the overall computing architecture is required. Finally, executing massive read/write block loops repeatedly over the host file system can accelerate the physical wear-and-tear of Flash memory endurance cells.

2.0 TOOLS INSTALLATION AND CONFIGURATION

2.1 Details on Installation and Configuration of Tool 1

2.1.1 Tool Overview

Cinebench 2026 is a free benchmarking application that has been created by Maxon. It is used to measure the performance of computer CPUs by rendering an incredibly complicated 3D graphic with the help of the Redshift renderer that is used in production. This application stresses the CPU pipelines, ALUs, and FPUs to the maximum extent possible. Cinebench 2026 was designed to analyze the speed of performing intensive operations on a particular processor and its response to thermal restrictions and clock variations.

2.1.2 Installation and Environment Setup

1. Download the installer:

Go to the official [Cinebench 2026 Download Page](#) and click the dropdown menu to download the installer package by choosing the first option (CINEBENCH 2026 FOR WINDOWS X86_64).

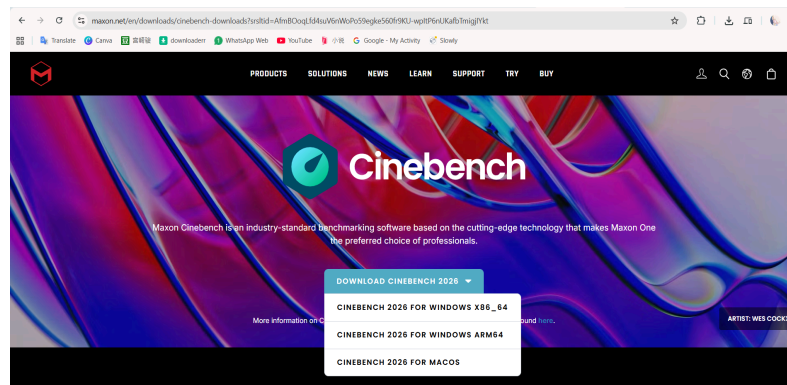


Figure 2.1.3.1 Cinebench 2026 Download Page

2. Extract the File:

- Locate the downloaded compressed folder (.zip) in the "Downloads" directory, right-click on the zip archive to open the context menu, then scroll down and click on Extract All as shown in Figure 2.1.3.2.

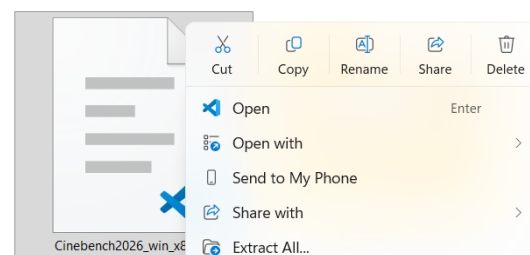


Figure 2.1.3.2 Right-click the zip file and select Extract All...

- When the extraction window pops up, browse to the dedicated project path in the destination box, and click the Extract button at the bottom of the window as shown in Figure 2.1.3.3.

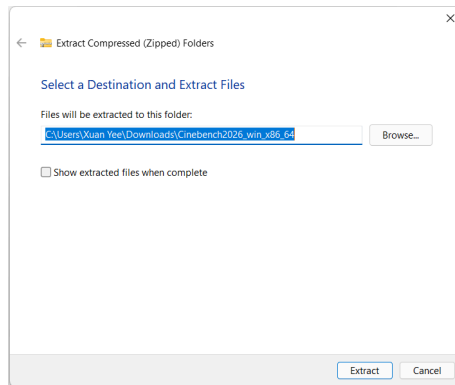


Figure 2.1.3.3 Destination Folder

4. Launch the Executable:

Double-click the application file to start the program as shown in Figure 2.1.3.4.

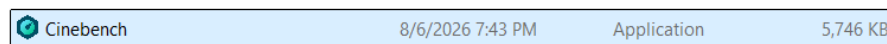


Figure 2.1.3.4 Cinebench 2026 Application File

2.1.3 Brief Operational Manual for Execution

1. Unlock Advanced GUI Controls:

Launch Cinebench 2026.exe. Click the File menu at the top left of the interface and check Advanced Benchmark to unlock the custom loop run options.

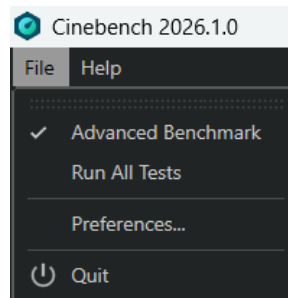


Figure 2.1.3.1 Enabling Advanced Benchmark settings in the File menu

2. Configure Workload A (Multi-Core Test):

- Click the dropdown selection field and select the 30 minutes (Test Stability) option as shown in Figure 2.1.3.2. With this option, the benchmark will enter an endless loop that lasts for 1800 seconds.

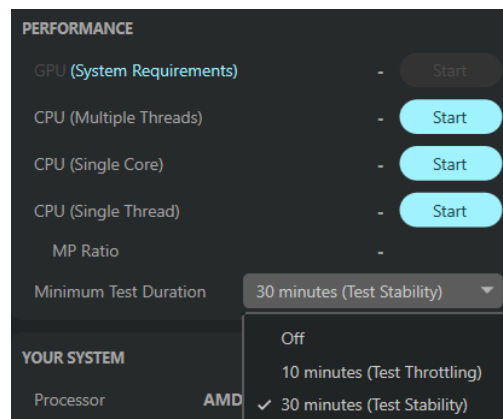


Figure 2.1.3.2 Select the 30 minutes (Test Stability) parameter from the dropdown options

- Point the mouse cursor upwards to reach the CPU (Multiple Threads) line and click on the start button next to it. This will simultaneously activate all the logical cores' pipelines to conduct the artificial stress testing process.

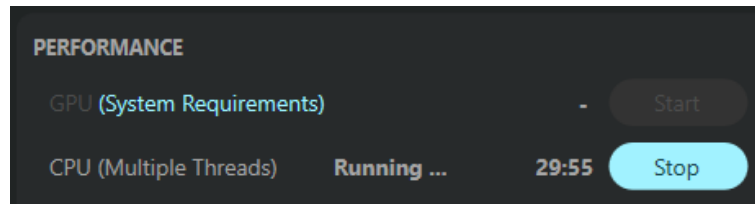


Figure 2.1.3.3 Executing the multi-threaded benchmarking sequence to begin the 30-minute system stress analysis.

3. Configure Workload B (Single-Core Test):

- Locate the minimum test duration dropdown menu next to CPU (Single-Core). Set the duration to 10 minutes to isolate a single execution thread.

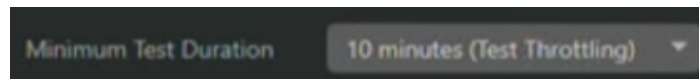


Figure 2.1.3.4 Select the 10 minutes (Test Throttling) parameter from the dropdown options

- Move the mouse cursor to the row of CPU (Single Core) and click on the start button. The above step will confine the benchmark performance testing to only one core pipeline.

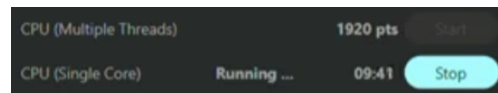


Figure 2.1.3.5 Executing the single-core benchmarking sequence to begin the 10-minute microarchitectural efficiency analysis

2.2 Details on Installation and Configuration of Tool 2

2.2.1 Tool Overview

HWiNFO64 is an industry-standard, professional hardware diagnostic and real-time monitoring sub-system engineered for x86 and x64 operating system environments. Within the scope of Computer Organization and Architecture (COA) evaluation, HWiNFO64 functions as a low-level microarchitectural telemetry agent. Unlike high-level application benchmarks that output abstract synthetic scores, HWiNFO64 interfaces directly with hardware-level registers, Model-Specific Registers (MSRs), and on-chip Digital Thermal Sensors (DTS) distributed across the system topology. For this project, HWiNFO64 is uniquely paired with computational stressors (such as Cinebench) to capture the immediate physical and electrical consequences of intense execution workloads on the underlying silicone.

2.2.2 Installation and Environment Setup

1. Download and Installation Phase:

The portable version of HWiNFO64 was downloaded from the official website which is [HWiFO64 Download Page](#) to avoid administrative privileges. Figure 2.2.2.1 shows the download page source. The Cinebench 2026 was downloaded refer to 2.1.2 Installation and Environment Setup



Figure 2.2.2.1 HWiNFO64 download page

2. Extract the Zip Archive Phase:

Since this is a portable version, no installer is required. The ZIP file [zip file](#) was extracted using Windows built-in extraction tool:

- Right click on hwi_834.zip
- Select “Extract All...” from context menu that shown in Figure 2.2.2.2
- Choose a destination folder (any preferred location) shown in Figure 2.2.2.3 and click extract

The zip file is extracted to the destination folder.

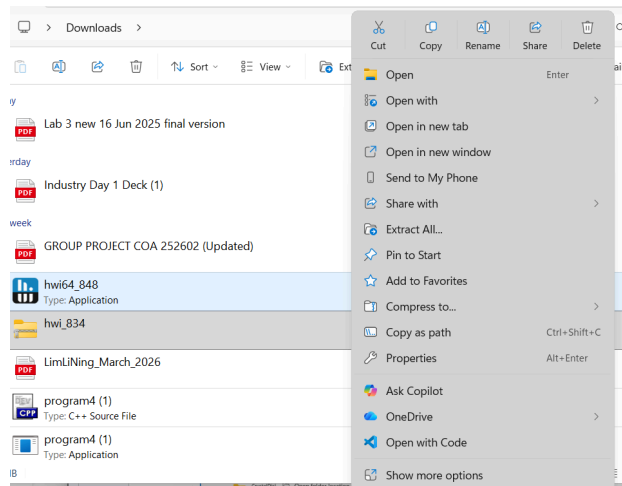


Figure 2.2.2.2 Screenshot extraction file

Extract Compressed (Zipped) Folders

Select a Destination and Extract Files

Files will be extracted to this folder:

C:\Users\limni\Downloads\hwi_834

Browse...

☒ Show extracted files when complete

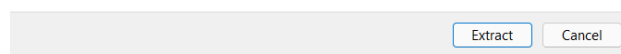


Figure 2.2.2.3 Screenshot destination folder

3. Launch the Correct Executable

After extraction, three executable files are present in the folder shown in figure 2.2.2.4.

- HWINFO64.exe (for 64-bit Windows)
- HWINFO32.exe (for 32-bit Windows)
- HWINFO_ARM64.exe (for ARM-based devices)

Since the test system runs 64-bit Windows, the correct file to launch is HWINFO_ARM64.exe.

	HWINFO_ARM64	5/6/2026 23:50	Application	12,911 KB
	HWINFO32	5/6/2026 23:50	Application	3,235 KB
	HWINFO64	5/6/2026 23:50	Application	9,456 KB

Figure 2.2.2.4 Three executable files

2.2.3 HWiNFO64 Sensor Configuration

1. Select Sensors-Only Mode Phase:

Upon startup, the HWiNFO64 system information dialog appears. The "Sensors-only" checkbox was selected to enable real-time hardware monitoring shown in Figure 2.2.3.1.

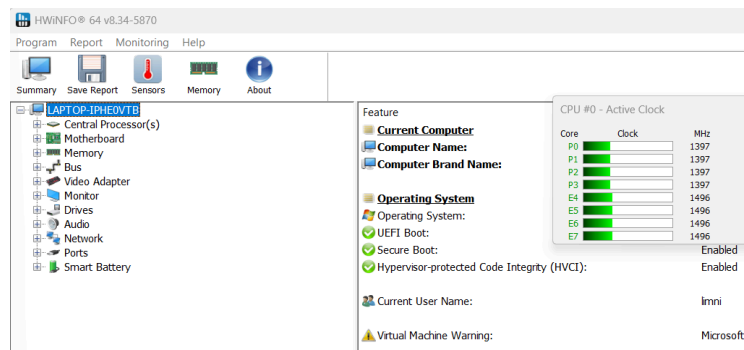


Figure 2.2.3.1 Sensors icon

2. Configure Global Polling Interval

Once the sensor window opens, the polling interval was configured as follows:

- Click the gear icon (Settings) at the bottom-right corner of the sensor window
- In the setting dialog, locate "Global Polling Period"
- Set the value to 2000 ms (2 seconds) that shown in Figure 2.2.3.2
- Click "Set" then "OK" to confirm

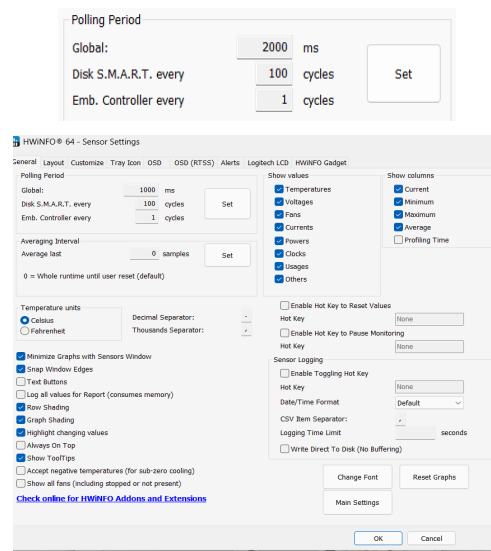


Figure 2.2.3.2 Polling Period

3. Workload Sampling Configuration

- Workload A Sampling Configuration (Steady- State Thermal Capture):
 - ❖ Target Selection: Row 1(Core Thermal Status, Clock Frequencies, and Platform Power Domain) shown in Figure 2.2.3.3
 - ❖ Sampling Polling Interval (I_A): Configured to 1000ms to prevent data buffer saturation across the long-duration test cycle.
 - ❖ Telemetry Data Windows: Synchronized to a continuous 30-minute execution envelope to isolate steady-state Heat-Dissipation curves.
 - ❖ Microarchitectural Metrics Tracked:
 - ☐ Core Thermal Throttling [Yes/No] Binary pipeline execution degradation flags).
 - ☐ CPU Package Power (W) (Dynamic electrical power consumption registers).
 - ☐ Core Clocks (MHz) (Real-time instruction pipeline frequency shifts).
 - ❖ Execution Constraint: Restrained to macroscopic steady-state tracking to capture long-term Clock Frequency Deltas ($\Delta\tau$) without introducing high storage I/O logging overhead.
- Workload B Sampling Configuration (High-Frequency Microstructural and Thread Shifting Tracking):
 - ❖ Target Selection: Row 2(Individual Core Utility, Effective Clocks, and Thread Scheduling Domain) shown in Figure 2.2.3.3
 - ❖ Sampling Polling Interval (I_A): Compressed downwards to 500 ms (exactly 0.5 seconds per snapshot loop) to double the telemetry sampling density to 2Hz.
 - ❖ Telemetry Logging Envelope: Synchronized to a strict 10 minute execution runtime, yielding a time-series dataset to approximately 1200 captured rows.
 - ❖ Microarchitectural Metrics Tracked:
 - ☐ Effective Clock Frequencies (Actual executing clock rate per individual thread line).
 - ☐ Individual Core Temperatures (Localized structural junction thermal hotspots)
 - ☐ Core Utility Flags (Thread migration footprints across Performance and Efficient cores)
 - ❖ Execution Constraint: Scaled upwards in frequency to prevent temporal aliasing, ensuring that every rapid dynamic thread context switch executed by the Windows thread scheduler across the physical cores of the Intel Core i5-13420H processor is cleanly registered.

CPU [#0]: Intel Core i5-13420H				
> ⚡ Core VIDs	0.690 V	0.629 V	0.956 V	0.687 V
⚡ Uncore VID	0.621 V	0.591 V	0.848 V	0.621 V
⚡ SA VID	0.661 V	0.658 V	0.813 V	0.674 V
> ⌚ Core Clocks	1,446.5 MHz	897.8 MHz	4,090.0 MHz	1,428.0 MHz
⌚ Bus Clock	99.8 MHz	99.7 MHz	99.8 MHz	99.8 MHz
⌚ Ring/LLC Clock	798.0 MHz	398.9 MHz	3,092.4 MHz	787.2 MHz
> ⌚ Core Effective Clocks	281.6 MHz	0.2 MHz	2,585.7 MHz	306.9 MHz
⌚ Average Effective Clock	337.8 MHz	215.6 MHz	1,449.9 MHz	367.8 MHz
> ⌚ Core Usage	20.8 %	0.0 %	81.2 %	24.4 %
⌚ Max CPU/Thread Usage	65.4 %	45.1 %	81.2 %	62.4 %
⌚ Total CPU Usage	20.8 %	14.3 %	49.9 %	24.4 %
⌚ On-Demand Clock Modulation	100.0 %	100.0 %	100.0 %	100.0 %
> ⌚ Core Utility	14.5 %	0.0 %	128.8 %	15.9 %
⌚ Total CPU Utility	14.5 %	9.4 %	62.2 %	15.9 %
> ⌚ Core Ratios	14.5 x	9.0 x	41.0 x	14.3 x
⌚ Uncore Ratio	8.0 x	4.0 x	31.0 x	7.9 x

Figure 2.2.3.3 HWiNFO64 active sensor grid capturing the real-time execution parameters

2.2.4 Brief Operational Manual for Execution

1. Launch HWiNFO64 in Sensors-Only Mode:

Double-click HWiNFO64.exe and ensure "Sensors-only" is checked before clicking "Run".

2. Start CSV Logging

Action	Icon/Location	Description
Start logging	Green[+] icon (located bottom-right)	Click to begin recording sensor data to CSV
Name the file	Dialog box appears	Enter filename
Choose save location	Browse folder	Select desktop or project folder for easy access

3. Run the Desired Workload

After logging has started:

- Minimize HWiNFO64 (do NOT close it)
- Launch Cinebench 2026 / R23 referred to tool 1 which is 2.1.3
- Execute the required workload (see Sections 3.1–3.3 for details)

4. Stop CSV Logging

After the benchmark finishes, stop logging by clicking the red[x] icon which is located on the bottom-right. Its' function is to stop recording and save the csv file.

5. Locate and verify the Log File

Navigate to the save location and verify:

- The CSV file exists with the correct filename
- File size is > 0 KB (contains data)
- Open in Notepad or Excel to check headers (e.g., Figure 2.2.4.2)

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
Date	Time	Virtual Mem	Virtual Mem	Physical Mem	Physical Mem	Physical Mem	Page File	Core	VIDs	P-core 0	P-core 1	P-core 2	P-core 3	P-core 4	P-core 5	P-core
8.6.2026	53:13.1	16086	5810	73.4	11919	4089	74.4	1.9	0.678	0.682	0.661	0.658	0.664	0.689	0.689	0.689
8.6.2026	53:15.1	16049	5847	73.2	11909	4099	74.3	1.9	0.691	0.664	0.662	0.664	0.671	0.721	0.717	0.717
8.6.2026	53:17.1	16043	5853	73.2	11887	4121	74.2	1.9	0.672	0.63	0.639	0.64	0.662	0.7	0.702	0.702
8.6.2026	53:19.1	16034	5862	73.2	11888	4120	74.2	1.9	0.687	0.654	0.666	0.663	0.663	0.713	0.713	0.713
8.6.2026	53:21.1	16030	5866	73.2	11884	4124	74.2	1.9	0.681	0.635	0.642	0.657	0.662	0.712	0.712	0.712
8.6.2026	53:23.1	16017	5879	73.1	11872	4137	74.1	1.9	0.685	0.645	0.665	0.661	0.661	0.716	0.711	0.711
8.6.2026	53:25.1	16018	5878	73.1	11872	4136	74.1	1.9	0.691	0.668	0.664	0.661	0.665	0.718	0.718	0.718
8.6.2026	53:27.4	16019	5877	73.1	11886	4122	74.2	1.9	0.691	0.668	0.666	0.666	0.666	0.714	0.715	0.715
8.6.2026	53:29.2	16029	5867	73.2	11896	4112	74.3	1.9	0.695	0.681	0.686	0.667	0.666	0.716	0.716	0.716
8.6.2026	53:31.1	16028	5868	73.1	11897	4111	74.3	1.9	0.685	0.662	0.66	0.66	0.661	0.72	0.71	0.71
8.6.2026	53:33.2	16046	5850	73.2	11911	4097	74.4	1.9	0.691	0.668	0.668	0.663	0.66	0.718	0.714	0.714
8.6.2026	53:35.5	16040	5856	73.2	11906	4102	74.3	1.9	0.687	0.664	0.659	0.657	0.662	0.713	0.713	0.713
8.6.2026	53:37.2	16045	5851	73.2	11902	4106	74.3	1.9	0.665	0.638	0.639	0.639	0.642	0.69	0.69	0.69
8.6.2026	53:39.1	16028	5868	73.1	11897	4112	74.3	1.9	0.69	0.669	0.662	0.663	0.663	0.708	0.715	0.715
8.6.2026	53:41.1	16025	5871	73.1	11897	4111	74.3	1.9	0.688	0.667	0.659	0.66	0.663	0.713	0.715	0.715
8.6.2026	53:43.1	16020	5876	73.1	11891	4117	74.2	1.9	0.688	0.666	0.66	0.66	0.659	0.711	0.714	0.714

Figure 2.2.4.2 CSV file in folder with Excel Preview

2.3 Details on Installation and Configuration of Tool 3

2.3.1 Tool Overview

An evaluation of computer performance and global comparisons are enabled by PassMark PerformanceTest 11.1 through the use of an indexed baseline database and standard units like MOps/Sec and GFLOPS. Although great convenience is provided by the software in measuring metrics like MIPS and reporting results, several challenges are presented. Direct comparisons between the overall score and real-world performance results cannot easily be made, score inconsistencies are produced between different versions (such as Version 11 versus Version 10), and specialized licensing is still required before network and advanced GPU tests can be performed.

2.3.2 Installation and Environment Setup

1. Open a web browser and navigate to the official [PassMark Download Page](#).

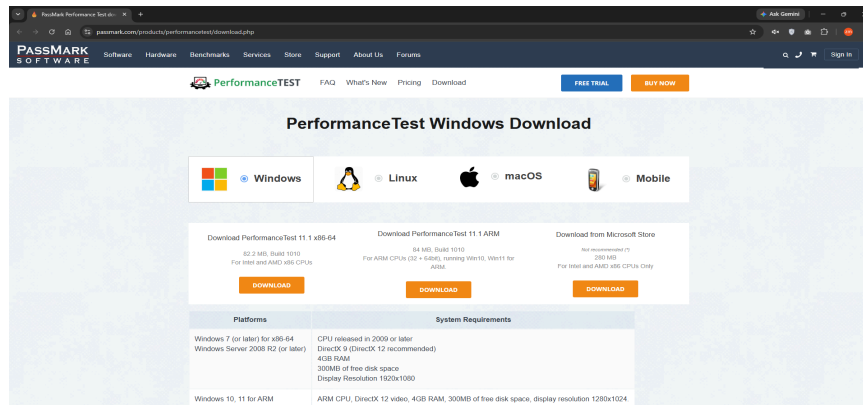


Figure 2.3.2.1 Official PassMark download page

- Click the "Download Now" button under the Windows Edition section. The installer file (named PerformanceTest_Windows_x86-64.exe) will begin downloading to your system's Downloads folder.

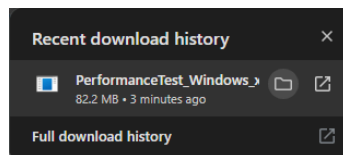


Figure 2.3.2.2 Installer file downloaded

- Locate the downloaded installer file in your Downloads folder and double-click it to launch the Setup Wizard.

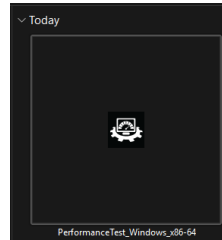


Figure 2.3.2.3 Installer file in download folder

- Read the License Agreement. Select "I Agree" to accept the terms and conditions, then click "Next" to continue.

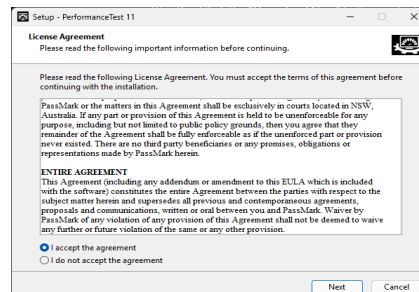


Figure 2.3.2.4 License Agreement page

- Choose the installation destination folder. The default path is: C:\Program Files\PerformanceTest\. It is recommended to leave this as default unless a different drive is preferred. Click "Next" to continue.

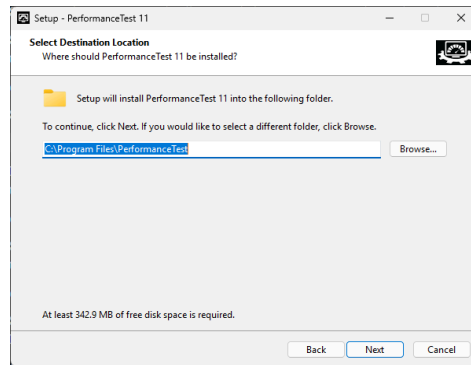


Figure 2.3.2.5 Installation destination folder

6. Click "Install" to begin the installation process. A progress bar will appear showing the installation status.
7. Once installation is complete, click "Finish" to exit the Setup Wizard. PassMark PerformanceTest will launch automatically.

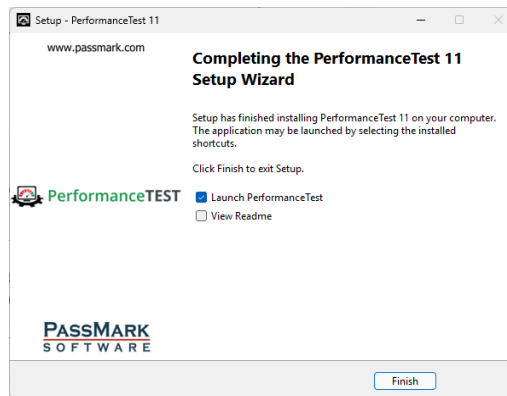


Figure 2.3.2.6 Installation completed

2.3.3 Brief Operational Manual for Execution

1. Upon first launch, PassMark PerformanceTest opens to the main dashboard. The left sidebar displays the available test categories:
 - PASSMARK (Overall Rating)
 - CPU MARK
 - 2D MARK
 - 3D MARK
 - MEMORY MARK
 - DISK MARK

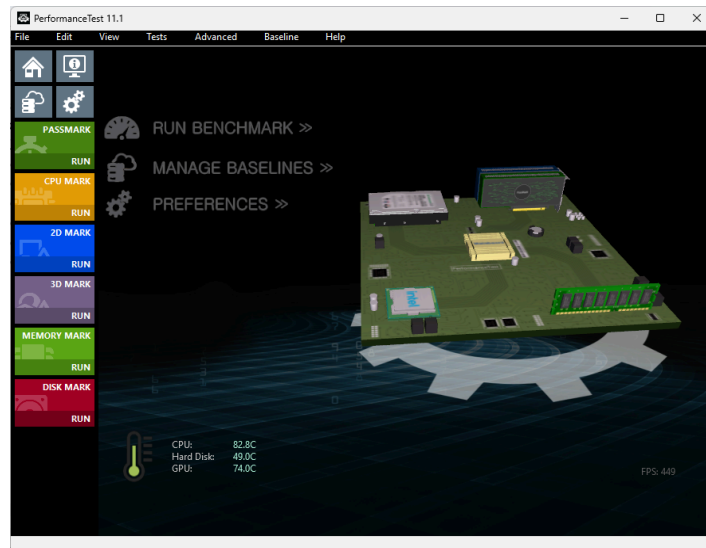


Figure 2.3.3.1 Main dashboard of PassMark PerformanceTest

2. For this project, click on "CPU MARK" in the left sidebar to access the CPU benchmarking module. This will display the individual CPU sub-tests including Integer Math, Floating Point Math, Sorting, Extended Instructions (SSE), Compression, Encryption, and Physics.

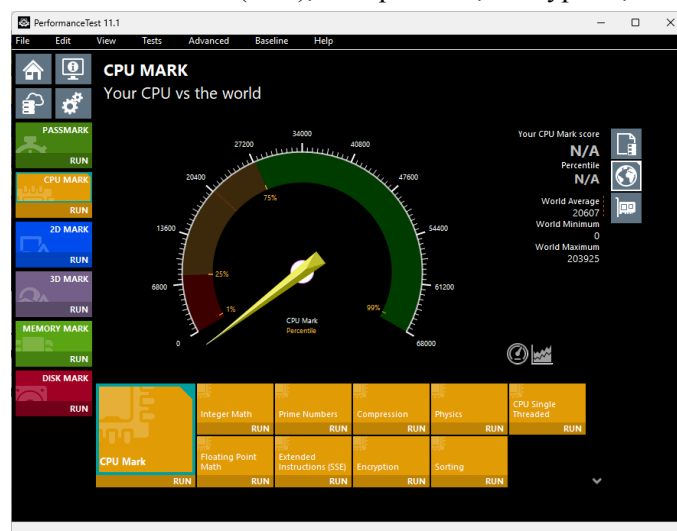


Figure 2.3.3.2 CPU MARK dashboard

2.4 Details on Installation and Configuration of Tool 4

2.4.1 Tool Overview

CrystalDiskMark is an open source disk drive benchmark tool for Microsoft Windows from Crystal Dew World. Based on Microsoft's MIT-licensed Diskspd tool, this graphical benchmark is commonly used for testing the performance of solid-state storage. It works by reading and writing through the filesystem in a volume-dependent way. It generates read/write speeds in sequential and random positions with varying numbers of queues and threads. Solid-state drives tend to excel at random IO, as unlike hard drives, they do not need to seek for the specific position to read from or write to. For this project, CrystalDiskMark is selected to evaluate Memory Bus Controller Overhead and DMA vs. Interrupt I/O handling by analyzing throughput scores (MB/s) and IOPS under different queue depth configurations as specified in Workload A and Workload B.

2.4.2 Installation and Environment Setup

1. Download CrystalDiskMark from the [CrystalDiskMark Download Page](#). Click Download Installer based on the CrystalDiskMark Standard. Figure 2.4.2.1 shows the download source page.

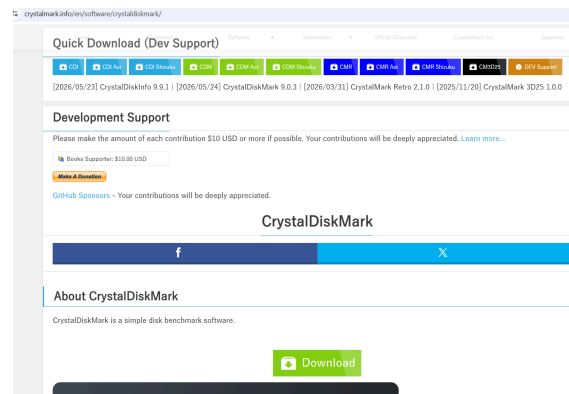


Figure 2.4.2.1 CrystalDiskMark download page

2. Extract the ZIP file to a target directory (e.g., C:\XXX\CrystalDiskMark). Click “Extract All...” Figure 2.4.2.2 shows the extraction of the ZIP file.

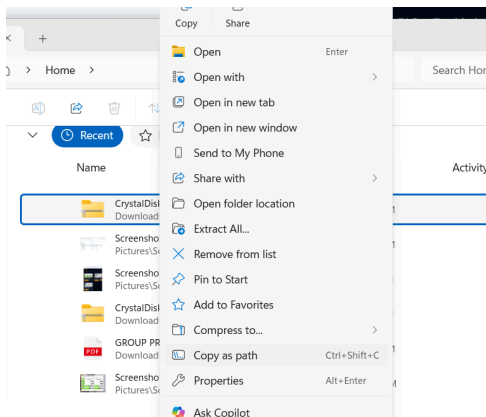


Figure 2.4.2.2 Extraction ZIP file

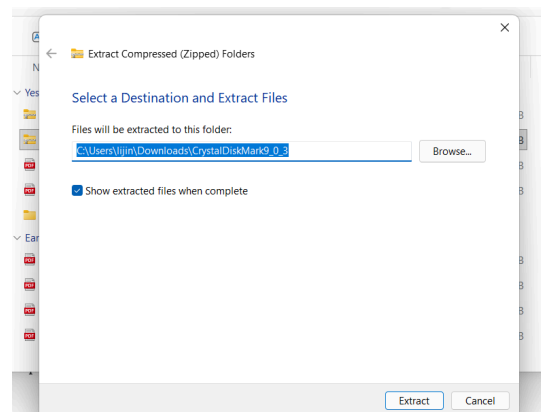


Figure 2.4.2.3 Destination folder

3. Open the extract folder.

4. Based on the operating system architecture, double click DiskMark64.exe(for 64-bit systems) or DiskMark32.exe (for 32-bit systems) to launch the tool. Figure 2.4.2.4 shows the tool to launch DiskMark. Figure 2.4.2.5 shows the tool after launching.

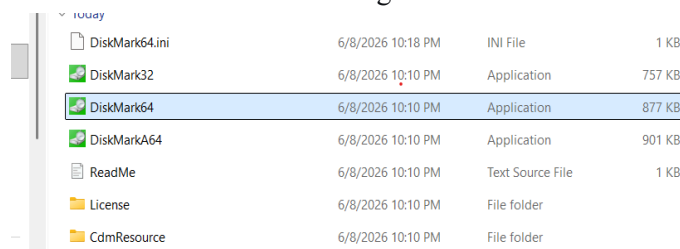


Figure 2.4.2.4 Launch the tool

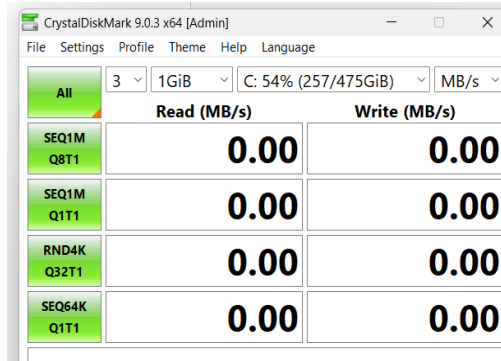


Figure 2.4.2.5 The screen after opening the tool

Environment Configuration Notes:

- To ensure measurement accuracy, all unnecessary background applications (e.g., antivirus software, file synchronisation tools, and browser tabs) were closed before launching the benchmark. This minimises CPU and memory bus interference.
- The target drive was selected from the dropdown menu in the CrystalDiskMark interface. For this project, the OS drive (C:) was tested.

2.4.3 Workload Parameter Configuration Matrix

To isolate the microarchitectural attributes specified in the project charter for Workload A and Workload B, the default testing profile registers were adjusted within the memory abstraction layer. The configuration was implemented via the Settings menu by launching the Queues / Threads dialog window. Figure 2.4.3.1 shows the setting screen.

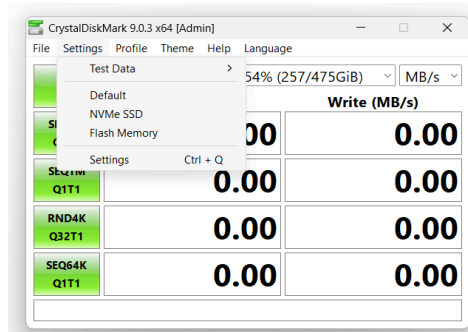


Figure 2.4.3.1 Setting screen

The custom operational profiles were mapped under the Profile: Default block as follows:

- Workload A Configuration (Random I/O Controller Stress):
 - **Target Selection:** Row 3 (Random Read/Write Domain).
 - **Block Size:** Configured to 4KiB to simulate highly fragmented block transactions.
 - **Queue Depth (Q):** Scaled upwards to 32 to fully saturate the hardware command register lines.
 - **Thread Count (T):** Restrained to 1 to isolate core storage controller handling capability without multi-threaded scheduling overhead.

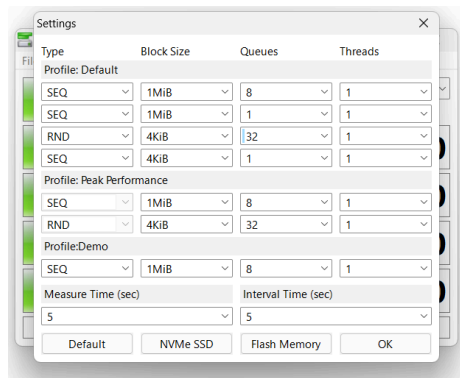


Figure 2.4.3.2 Row 3 change

- Workload B Configuration (Sequential Data Loading Emulation):
 - **Target Selection:** Row 4 (Modified from Random to Sequential).
 - **I/O Type (Type):** Reconfigured from RND to SEQ via the drop-down selection panel.
 - **Block Size (Block Size):** Scaled to 64KiB to enable continuous block sequential streaming.
 - **Queue Depth (Q):** Bound to 1 to eliminate hardware native command sorting optimizations.
 - **Thread Count (T):** Assigned to 1 to observe raw single-channel un-burst memory execution bounds.

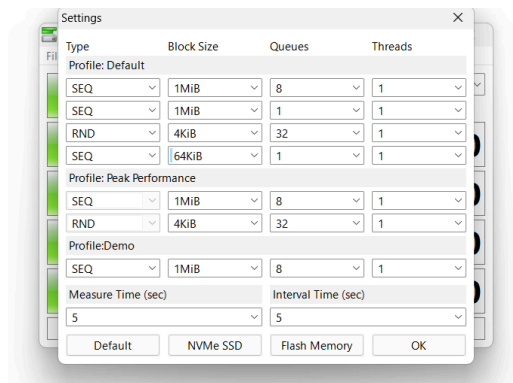


Figure 2.4.3.3 Row 4 change

2.4.4 Brief Operational Manual for Execution

Once the internal microarchitectural parameters were successfully initialized, the benchmark was executed based on the following standard operational guidelines:

1. **Test Parameter Initialization:** On the main dashboard tool-bar, the global test loop count was set to 3 (or 5), and the standard target file block size was locked to 1GiB.
2. **Volume Selection Target:** The active logical drive selection was restricted to the primary operating system volume partition (C: Drive) to guarantee direct block-level system bus load generation.
3. **Execution Routine:** The localized execution sequence was committed by clicking either the individual target row buttons or the primary green ALL execution register on the top-left section of the user interface.
4. **Data Extraction Method:** Upon termination of the simulation iterations, the data lines representing RND4K Q32T1 (Throughput MB/s and IOPS) and SEQ64K Q1T1 (Linear transfer

speed in MB/s) were logged, and the primary diagnostic graphical display was captured for technical analysis.

3.0 PERFORMANCE BENCHMARK ANALYSIS AND DISCUSSIONS

3.1 Detailed Explanation on Workload A and Workload B

a. Tool 1 - Cinebench 2026 / R23

- **Workload A - Multi-Core Execution Framework (30-Minute Custom Loop):**

This benchmark suite was created to test the multi-threading capacity of the processor, its hardware scheduler efficiency, and the thermal stability at maximum stress. During the running of the CPU (Multiple Threads) benchmark application, the Redshift renderer divides image data into smaller micro-tasks and processes them using all available physical and virtual cores through simultaneous multithreading (SMT). In a multi-core execution environment, all available execution pipelines will be saturated at the same time. A 30-minute custom loop is an important architecture analysis tool as it enables the processor to run beyond its initial burst-frequency period and stabilize at a constant state of thermal equilibrium. It is an effective way for one to observe and analyze the process of pipeline saturation efficiency, structural clock-speed decrease, and how the OS thread scheduler spreads heavy mathematical computations across the die as heat builds up.

- **Workload B: Single-Core Execution Framework (10-Minute Custom Loop)**

Unlike Workload A, Workload B focuses on a single processing pipeline by confining the benchmark execution solely within the Single Core (CPU) domain. In this scenario, only one logical thread is involved for completing the rendering process. By doing so, it becomes easier to strip away the complexities associated with multi-threaded overheads, including inter-core communication delays, competition among threads, and L3 cache contention, thus creating an ideal setup for measuring Instruction Per Clock cycle efficiency of a microarchitectural core. In addition, the throttling window is reduced to 10 minutes in case of Workload B. The reason behind this arrangement is that unlike in the previous scenario, a single core-based workload causes intense localized thermal activity confined to only a small portion of the silicon die. The 10 minute loop enables the evaluation team to study the dynamic thermal behavior, particularly thread migrations caused by the operating system, in an attempt to avoid localized hot spots while boosting clock speeds to their maximum.

b. Tool 2 - HWiNFO64

- **Workload A - Multi-Core Execution Framework (30-Minute Custom Loop):**

This workload is designed to evaluate the processor's multi-threading capacity and thermal stability under sustained full-core saturation. By forcing a continuous 30-minute rendering loop across all available physical and logical cores using Cinebench 2026, the test measures pipeline saturation efficiency, structural clock frequency degradation (ΔT), and thermal throttling behavior when the processor exceeds its sustained power budget. HWiNFO64 captures these microarchitectural events by logging the Core Thermal Throttling [Yes/No] flag to detect binary pipeline execution stalls, comparing Core Clocks [MHz] against Base Clock [MHz] to calculate the frequency delta during throttling, and recording CPU Package Power [W] to correlate power draw reduction with thermal limiting events. The 30-minute duration forces the processor beyond its initial burst-frequency window into a steady-state thermal equilibrium, revealing how the OS thread scheduler spreads heavy computations across the die as heat accumulates.

- **Workload B: Single-Core Execution Framework (10-Minute Custom Loop) :**

This workload is designed to evaluate the processor's single-core IPC efficiency and thermal density management under localized stress. By confining the benchmark to a single logical core using Cinebench 2026 Single-Core test, the test eliminates multi-threaded complexities such as inter-core communication delays, thread contention, and L3 cache coherency traffic, creating an ideal environment for measuring raw per-core instruction throughput. HWiNFO64 captures single-core architectural behavior by tracking Core X Usage [%] to identify dynamic thread migration patterns (where the Windows scheduler shifts the active thread across cores every 4–6 seconds to prevent localized hotspots), monitoring Core X Temperature [°C] to measure the rapid heating (25–30°C rise) of the active core versus idle neighbors, and recording Core X Clocks [MHz] to verify whether maximum boost frequency is sustained. The polling interval is compressed to 500 ms to ensure rapid thread context switches are not lost to temporal aliasing.

- c. Tool 3 - PassMark PerformanceTest

- **Workload A (CPU Instruction Suite):**

PassMark PerformanceTest executes its standard CPU Instruction Suite across three sub-tests: Integer Math, Floating Point Math, and Sorting, which stress the ALU, FPU, and memory interface pipeline respectively. The test runs under default multi-threaded conditions, utilizing all available logical cores to simulate sustained, real-world instruction throughput pressure. The primary COA metrics extracted are MIPS (derived as Operations per Second $\div 10^6$ from the Test Results Log), CPI (estimated using the formula $CPI = \text{Clock Frequency (Hz)} \div (\text{MIPS} \times 10^6)$), and ISA Structural Bottlenecks (identified by comparing scores across the three sub-tests). Together, these metrics expose key architectural behaviors including instruction-level parallelism limits, thermal throttling dynamics, and resource contention between execution units under sustained load.

- **Workload B (Floating-Point Math and Vector Math):**

PassMark PerformanceTest isolates and executes only the Floating Point Math and Vector Math portions of its CPU test suite to evaluate the processor's SIMD (Single Instruction, Multiple Data) execution capabilities, specifically targeting vector extensions such as AVX2 and AVX-512. Unlike Workload A's broad pipeline stress, this workload focuses narrowly on the width and efficiency of the processor's vector pipelines, with the key COA metrics being SIMD Efficiency and FPU Throughput, both measured through the GFLOPS (Giga Floating Point Operations Per Second) rating extracted from PassMark's advanced metrics log. These metrics reveal critical architectural behaviors such as AVX-512 frequency offsets under thermal constraints, FPU pipeline stalls caused by long-latency operations, and cache coherence limitations when large vector datasets exceed L1/L2 cache capacity — collectively reflecting how effectively the microarchitecture handles modern complex vector workloads beyond scalar integer execution.

- d. Tool 4 - CrystalDiskMark

- **Workload A (Random 4KiB Profile, Q=32, T=1):**

This workload is specifically engineered to expose structural bottlenecks linked to address translation routines and highly fragmented data paths. By throwing non-contiguous 4KiB workloads at the drive controller, the system is blocked from using its pre-fetch buffers. Instead, it must rely on deep hardware queues (Q=32) to force Native Command Queuing

(NCQ) into play, testing whether the controller can optimize command registers and use Direct Memory Access (DMA) channels to stream data directly over the peripheral bus without interrupting the CPU.

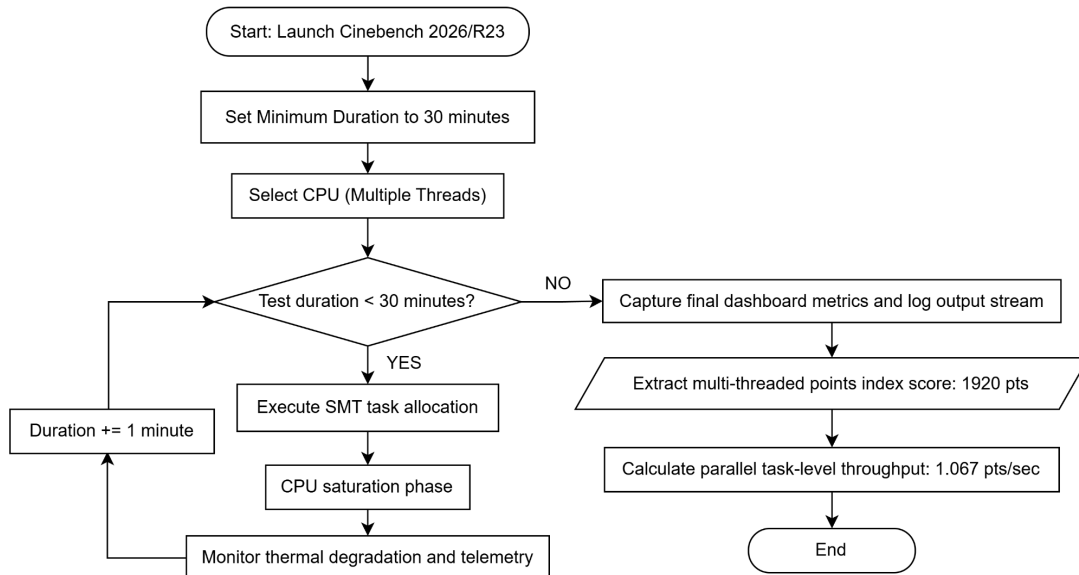
- **Workload B (Sequential 64KiB Profile, Q=1, T=1):**

This alternative configuration simulates loading raw, uncompressed memory blocks into the system. Because the 64KiB data streams are completely contiguous, the storage controller can push its Cache Line Fill Buffering limits to the max. However, by restricting the queue depth to a strict (Q=1), all hardware parallelization, multi-channel die interleaving, and command reordering are deactivated. This reveals the raw, Non-Burst System Bus Latency ceiling of an unassisted communication line.

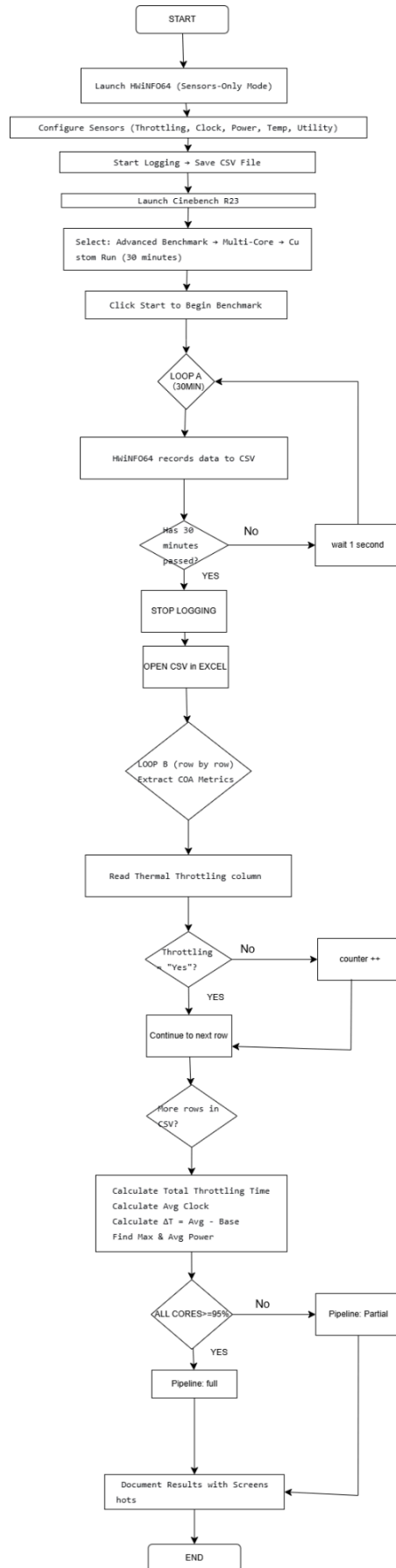
3.2 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload

A.

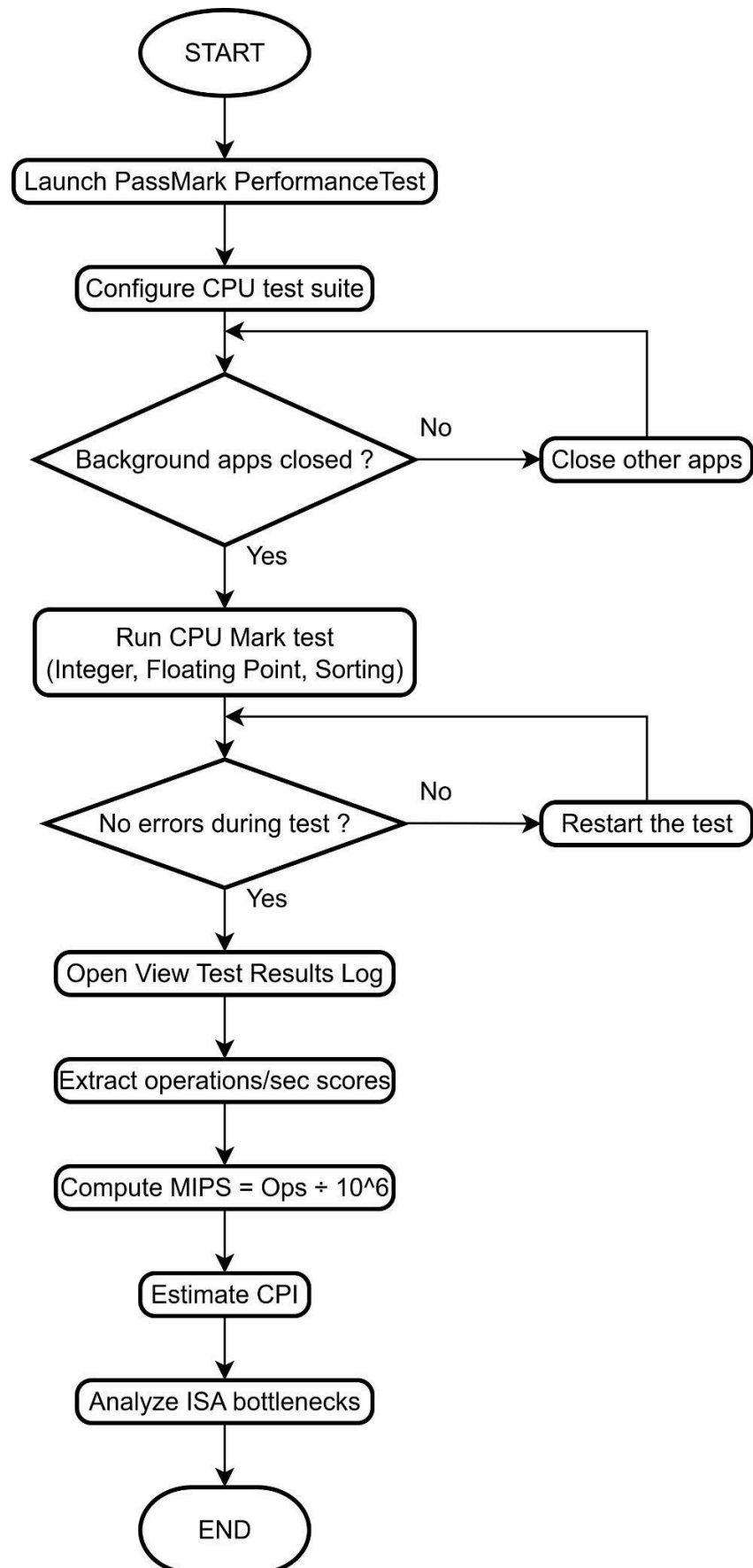
a. Tool 1



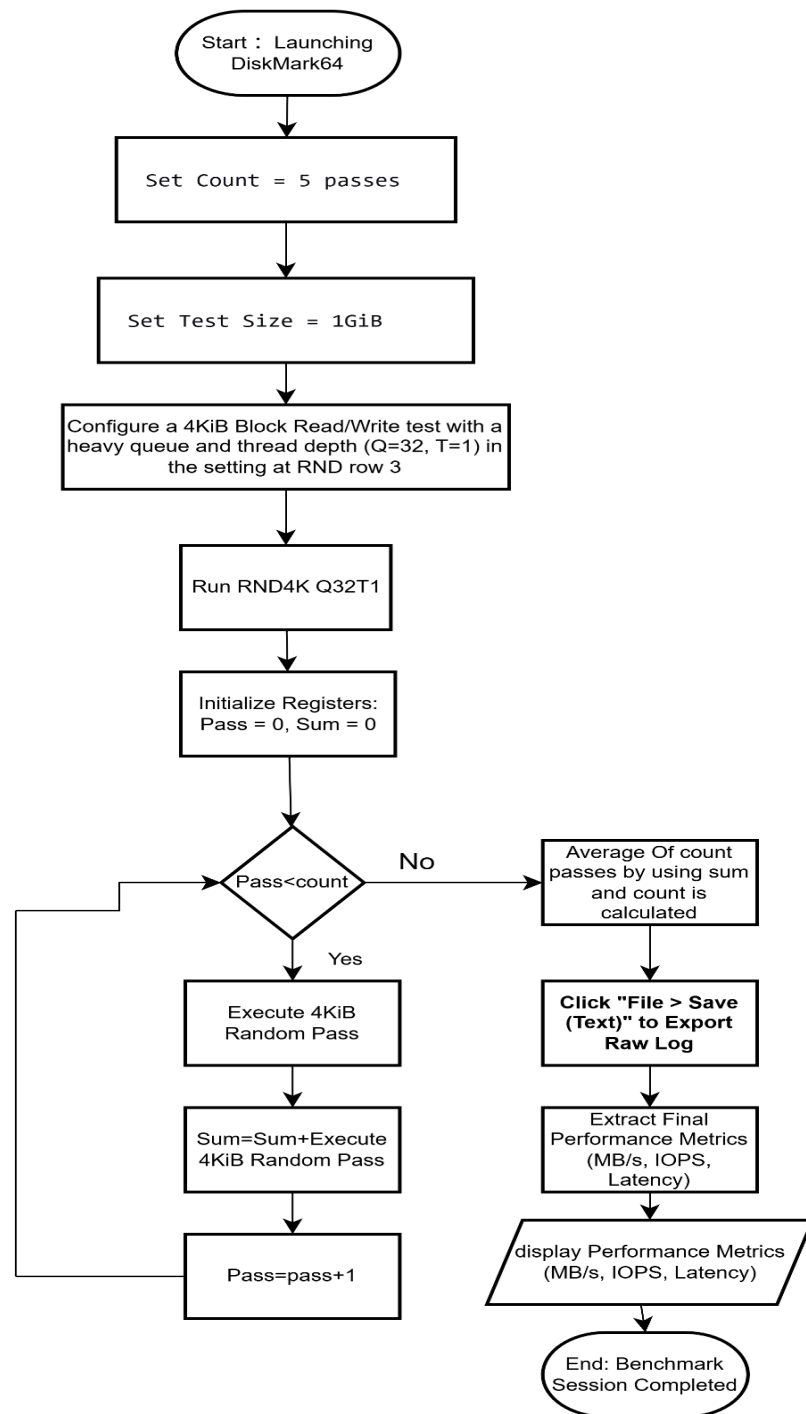
b. Tool 2



c. Tool 3

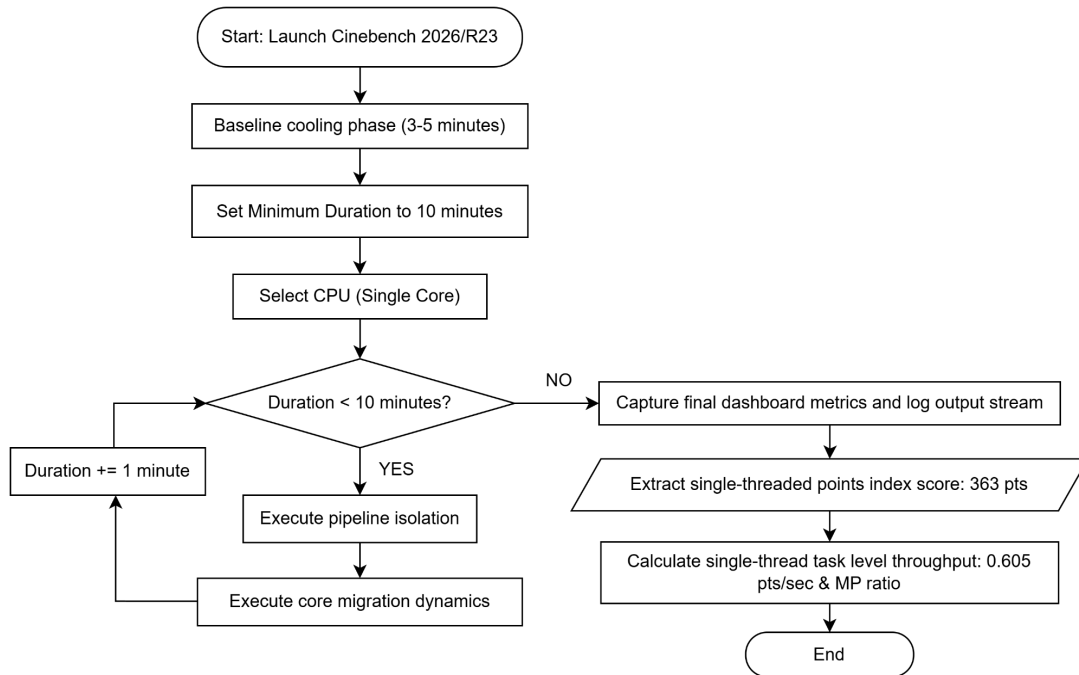


d. Tool 4

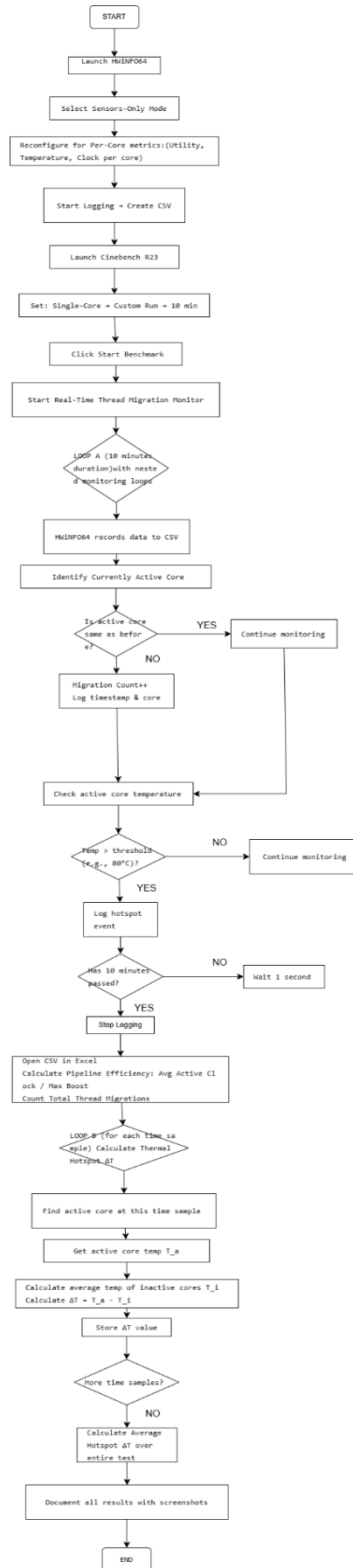


3.3 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload B.

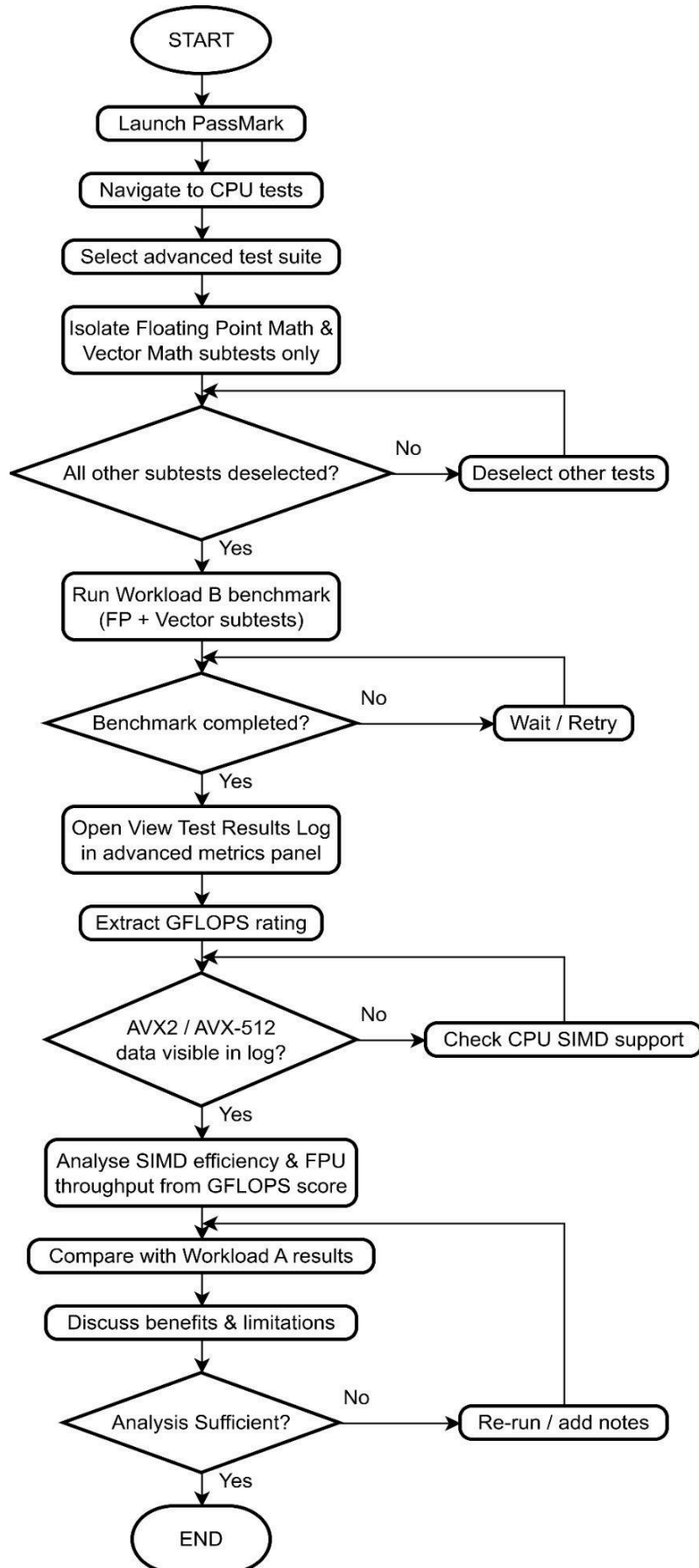
a. Tool 1



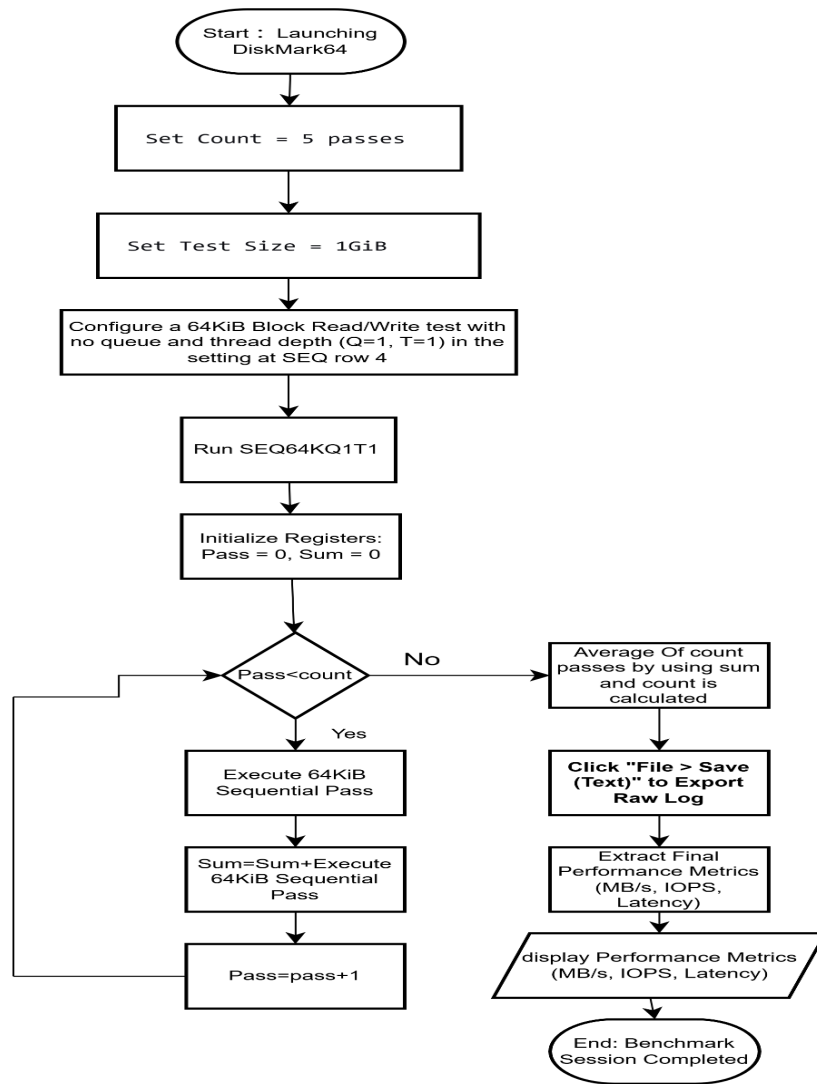
b. Tool 2



c. Tool 3



d. Tool 4



3.4 Analysis and Discussions

a. Tool 1

The empirical benchmark evaluation performed on the test system's AMD Ryzen 7 7730U processor architecture yielded the following performance metrics:

$$T_{cpu} = minutes \times 60 minutes/seconds$$

$$A: T_{cpu} = 30 \times 60 minutes/seconds = 1800 seconds$$

$$B: T_{cpu} = 10 \times 60 minutes/seconds = 600 seconds$$

$$Throughput_A = \frac{Multi - Core Score}{T_{cpu}} = \frac{1920 pts}{1800 pts} = 1.067 pts/sec$$

$$Throughput_B = \frac{Single - Core Score}{T_{cpu}} = \frac{363 pts}{600 pts} = 0.605 pts/sec$$

$$MP\ Ratio = \frac{Multi - Core\ Throughput\ Score}{Single - Core\ Throughput\ Score} = \frac{1920\ pts}{363\ pts}$$
$$= 5.289$$

Category	Workload A	Workload B
Test Configuration	30-Minute Multi-Core Loop	10-Minute Single-Core Loop
Throughput Score (Points Index)	1920 pts	363 pts
CPU Execution Time, T_{cpu}	1800 seconds	600 seconds
Task-Level Throughput (pts/sec)	1.067 pts/sec	0.605 pts/sec

By analyzing both metrics, it becomes apparent what kind of processing the computer architecture undertakes. Even though the general parallel processing capacity per multi-threaded parallel throughput of the processor is larger (1.067 pts/sec), the efficiency decreases over time in comparison to the optimal speed of single-threaded pipeline execution of 0.605 pts/sec, as seen in Figure 3.4.1 and Figure 3.4.2.

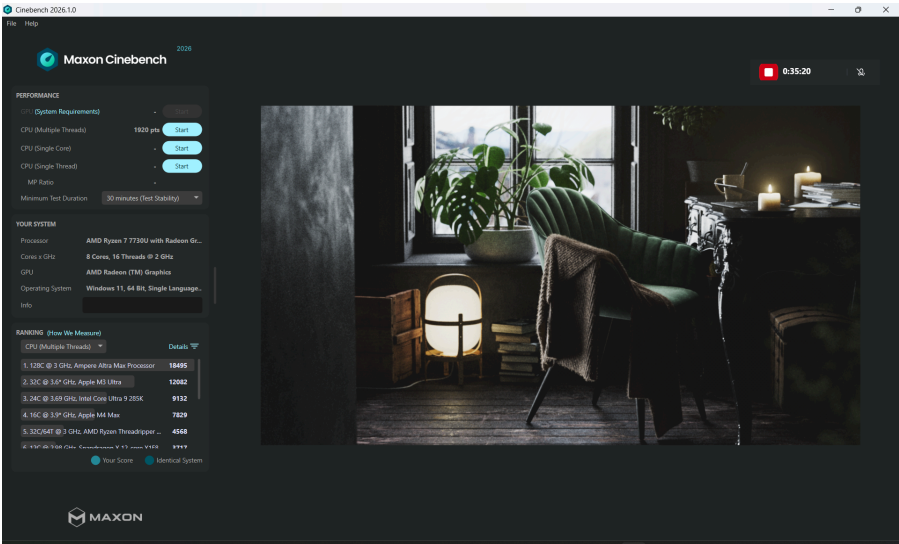


Figure 3.4.1 Workload A Multi-Threaded Score Dashboard (1920 pts)

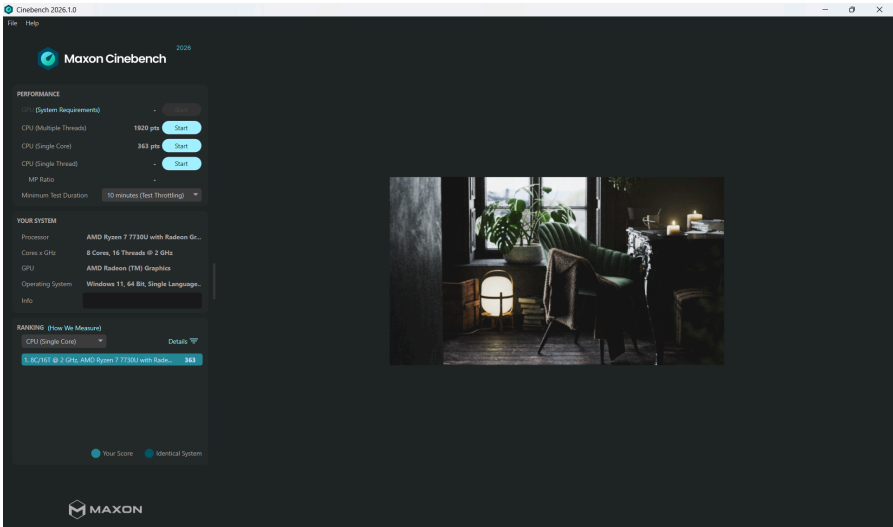


Figure 3.4.2 Workload B Single-Threaded Score Dashboard (363 pts)

It is evident that the comparison of the performances of the workloads Workload A and Workload B reflects a significant difference between the raw processing volume and the pipeline efficiency in isolation. Specifically, in case of the Workload A, the test managed to reach a higher total processing score, which amounts to 1920 points, as the code was executed simultaneously in all available physical and logical pipelines through the use of SMT technology. However, upon evaluating this test based on a per-second basis, it can be found out that the total throughput is 1.067 pts/sec because of the very heavy 30 minutes required for the task completion. In turn, workload B isolated one execution pipeline and managed to produce a total score of 363 points with a total throughput of 0.605 pts/sec in just 10 minutes. The MP Ratio in this case is 5.289. Ideally, on an 8-core processor, the MP ratio should theoretically be 8.0x.

Benefits

The empirical findings show high individual pipeline efficiency and good parallel instruction throughput for the test platform's microarchitecture. For Workload B (363 pts, generating 0.605 pts/sec), singling out one thread enabled the active core to use its individual execution resources optimally and attain maximum turbo boost frequencies, while the operating system scheduler allocated the task rotation between physical cores so as not to produce localized thermal hot spots. In switching to Workload A (1920 pts, generating 1.067 pts/sec), the hardware was capable of using SMT to execute the rendering matrix in all 16 logical execution pipelines simultaneously. Hardware-level parallelism reduced execution stage stalls by ensuring that other threads instantly occupied the available ALUs and FPU's when a certain thread stalled.

Limitations

Although the system exhibits high peak throughput, there is a clear trend of inefficiency due to sublinearity in the scaling factor, which is evident through an MP ratio of just 5.289 compared to an ideal theoretical ratio of 8.0. Such an inconsistency results from extreme limitations in terms of physical parameters within the system as imposed by the limited capabilities of the small and mobile hardware form-factor in dealing with high execution workload conditions. In the case of workload A in which all the cores operated at 100 percent in a loop for 30 minutes, the processor generated huge amounts of heat in excess of its cooling capacity. The system therefore invoked thermal safety measures by initiating thermal throttling, causing reduction of frequencies. Parallelism was further inhibited by the competition of 16 active threads for the same L3 cache blocks and memory bus bandwidth.

b. Tool 2

Workload A: Thermal Throttling, Clock Frequency Delta and Dynamic Power Consumption

This workload investigates the thermal behavior of the processor while Cinebench 2026 is running under sustained load. The analysis focuses on thermal throttling events, clock frequency variation and CPU power consumption.

Parameter	Value
Average Core Clock	1447.54 MHz
Maximum Core Clock	3591.2 MHz
Minimum Core Clock	1321.8 MHz
Average CPU Package Power	45.33 W

Maximum CPU Package Power	71.2 W
Average CPU Package	85°C
Maximum CPU Package	96°C
Thermal Throttling Event	8 occurrences

The log in figure 3.4.b.1 shows several entries where Core Thermal Throttling = YES, indicating that the processor reached a temperature or power condition requiring frequency reduction.

DD		EU	
1	Core Thermal Throttling (avg) [Yes/No]	1	CPU Package Power [W]
2	No	2	10.817
3	YES	3	45.28
4	YES	4	71.2
5	YES	5	65.8
6	YES	6	48.66
7	YES	7	46.8
8	No	8	44.5
9	No	9	45.298
10	No	10	45.29
11	No	11	45.29
12	No	12	45.29
13	No	13	45.29
14	No	14	45.29
15	No		
16	No		
17	No		

Figure 3.4.b.1 Core Thermal Throttling Figure 3.4.b.2 CPU Package Power[W]

DB						
1	CPU Package [°C]					
2	63					
3	96					
4	95.5					
5	94.8					
6	94					
7	85					
8	85					
9	85					
10	85					
11	85					
12	85					
13	85					

Sensor	Current	Minimum	Maximum	Average
CPU Package	86 °C	85 °C	96 °C	85 °C

Figure 3.4.b.3 CPU Package

Clock Frequency Delta ($\Delta\tau$) :

Using the recorded clock values shown in Figure 3.4.b.4 :

- Peak frequency = 3591.2 MHz
- Lowest frequency during workload = 1321.8 MHz

Sensor	Current	Minimum	Maximum	Average
> Core Effective Clocks	2811.4 MHz	1321.8 MHz	3591.2 MHz	1447.54 MHz

Figure 3.4.b.4 Core Effective Clocks

Therefore, $\Delta\tau = 3591.2 - 1321.8 = 2269.4$ MHz

This frequency reduction represents the processor's automatic response to increased heat generation.

These results indicated that the processor experienced a number of thermal throttling events during the Cinebench workload. At the beginning of the benchmark, CPU package power peaked at 71.2W,

which caused the temperatures to increase very rapidly. As thermal limits were approached, the processor reduced its operating frequency, resulting in a clock frequency drop of approximately 2269M Hz. Such thermal protection in modern computers' processors is common since it helps prevent overheating and ensures stable operation. However, the average temperature remained below the critical limit, suggesting that the cooling system was able to control the overall thermal condition after the initial power spike.

Conclusion:

The workload demonstrates that higher computational demand increases power consumption and temperature, this can lead to thermal throttling. The decrease in clock speed proves that the processor is regulating itself based on temperature changes.

Workload B: Dynamic Thread Migration and Localized Thermal Hotspots

This workload investigates how the Windows scheduler distributes a single heavy thread among processor cores during a Single-Core Cinebench loop to avoid localized overheating.

Parameter	Value
Average CPU Package Power	7.93 W
Maximum CPU Package Power	14.342 W
Average Core Temperature	53.72 °C
Maximum Core Temperature	61°C
Thermal Throttling Events	None

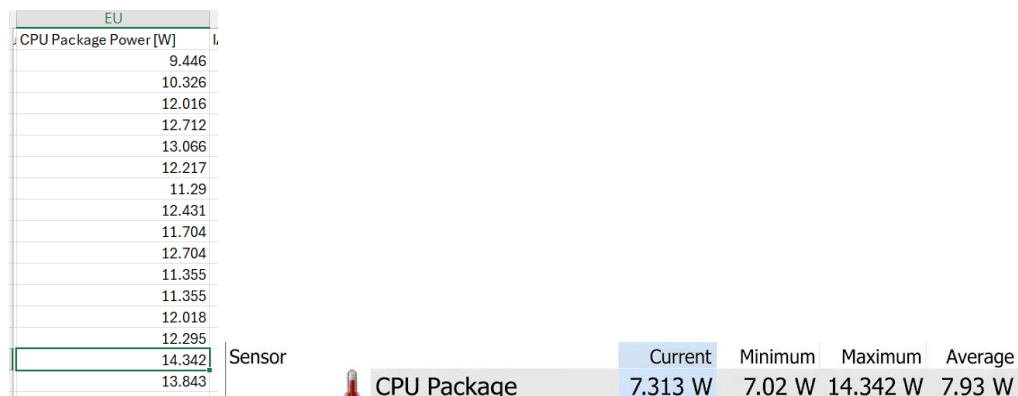


Figure 3.4.b.5 CPU Package Power shows the maximum, minimum and average

	DD
1	Core Thermal Throttling (avg) [Yes/No]
2	No
3	No
4	No
5	No
6	No
7	No
8	No
9	No
10	No
11	No
12	No
13	No
14	No
15	No
16	No
17	No

Figure 3.4.b.6 Core Thermal Throttling shows the none Thermal Throttling Events

Sensor	Current	Minimum	Maximum	Average
✓ CPU [#0]: Intel Core i5-13420H: DTS				
> 🔥 Core Temperatures	51 °C	40 °C	61 °C	53.72 °C

Figure 3.4.b.7 Core Temperatures shows that the max and average core temperature

Active logical threads:

Core Thread	Average Utility
P-core 0 T1	62.27%
P-core 0 T0	66.26%
P-core 1 T0	16.56%
E-core 7 T0	16.25%

The dominant active thread was observed mainly on:

- P-core 0 T0
- P-core 0 T1

With occasional movement to:

- P-core 0
- E-core 7
- E-core 6
- P-core 2

BN		BM		BO		BX	
P-core 0 T1 Utility [%]	P	P-core 0 T0 Utility [%]		P-core 1 T0 Utility [%]		E-core 7 T0 Utility [%]	
26.3		40.7	1	34.2	1	33.7	1
43.9		50.7	2	46.4	2	42.8	2
49.8		54.2	3	55.2	3	57.6	3
48.9		53.9	4	52	4	63.8	4
54.2		55.8	5	54.9	5	68	5
53.7		54.8	6	55.9	6	59.8	6
43.5		49.3	7	48.7	7	59.7	7
46.7		50	8	51.7	8	54.9	8
45		46.6	9	46.3	9	51.6	9
52.6		51	10	54.9	10	61.2	10
39.2		44.1	11	45.2	11	51.9	11
44.4		45.3	12	43.6	12	53.4	12
54.6		56.5	13	55	13	59.7	13
47.6		49.8	14	46.6	14	54.4	14
68		68	15	68	15	71.4	15
68		68	16	67.9	16	71.4	16
41.4		45.6	17	38.5	17	52.5	17
38.4		43.6	18	42.9	18	43.4	18
41.9		46.1	19	47.1	19	53.5	19
31.1		41.3	20	41.6	20	51.2	20
46.1		47.3	21	48.1	21	56.8	21
51.6		53.4	22	53.2	22	63.7	22
41.8		47.4	23	43.5	23	58.1	23
40.9		43.8	24	42.5	24	58.4	24
53.6		46.3	25	44.6	25	69.3	25
68		40.3	26	42.7	26	57.8	26
68		38.6	27	40.5	27	48.6	27
68		42.8	28	50.6	28	52.6	28
68		46	29	44.3	29	53.5	29
68		40.1	30	37.9	30	55.1	30
			31		31		31

As can be seen from the log, the Windows scheduler focused on executing tasks mainly on P-core 0, which is expected because Cinebench Single-Core testing attempts to maximize performance using one high-performance core. However, periodic activity appeared on other cores, which is clear evidence of dynamic thread migration. The scheduler moved the execution thread between physical cores to distribute thermal load and prevent excessive heat accumulation in a single region of the processor. The maximum recorded temperature was only 61°C and no thermal throttling events were observed so provided that thread migration effectively prevented localized thermal hotspots and kept the CPU running stable throughout the benchmark.

Conclusion:

The Cinebench single-core test clearly shows how the Windows scheduler dynamically shifts a heavy thread across different cores. By doing this, it reduces thermal density on any single core, prevents localized hotspots, and keeps processor performance perfectly stable without ever triggering thermal throttling.

Performance Comparison (Workload A vs Workload B):

Aspect	Workload A	Workload B
Workload type	High multi-core stress scenario	Single-core focused workload
Power Consumption	High (significantly increased under load)	Low (light and stable usage)
Temperature Behavior	High temperature, reaching near critical levels	Moderate temperature, remains controlled
Thermal Throttling	8 occurrences	None
Frequency Behavior	Unstable, frequent reduction due to heat	Stable, consistent performance
System Response	Hardware protection activated	OS scheduler optimization
Stability	Lower under heavy load	High and stable

Benefits

Use of the HWiNFO64 tool along with Cinebench 2026 brings some significant benefits when analyzing the performance of the CPU. First, HWiNFO64 provides real-time, granular monitoring, making it easy to track exact shifts in temperature, power draw, clock speeds, and throttling behavior. This lets us pin down exactly how the processor reacts when transitioning between heavy and light workloads. Second, it allows for a direct, side-by-side comparison of different stress scenarios, like Workload A and Workload B. We can easily contrast the heavy thermal throttling during the multi-core stress test against the stable, cool behavior during the single-core run. Third, the tool's per-core and thread-level tracking is invaluable for analyzing Workload B. It allows us to watch thread migration between P-cores and E-cores in real time, showing exactly how the Windows scheduler shuffles tasks to balance the thermal load. In conclusion, combining a performance benchmark, such as Cinebench 2026, with a monitoring tool, such as HWiNFO64, brings together the results of benchmark testing with the hardware conditions.

Limitations

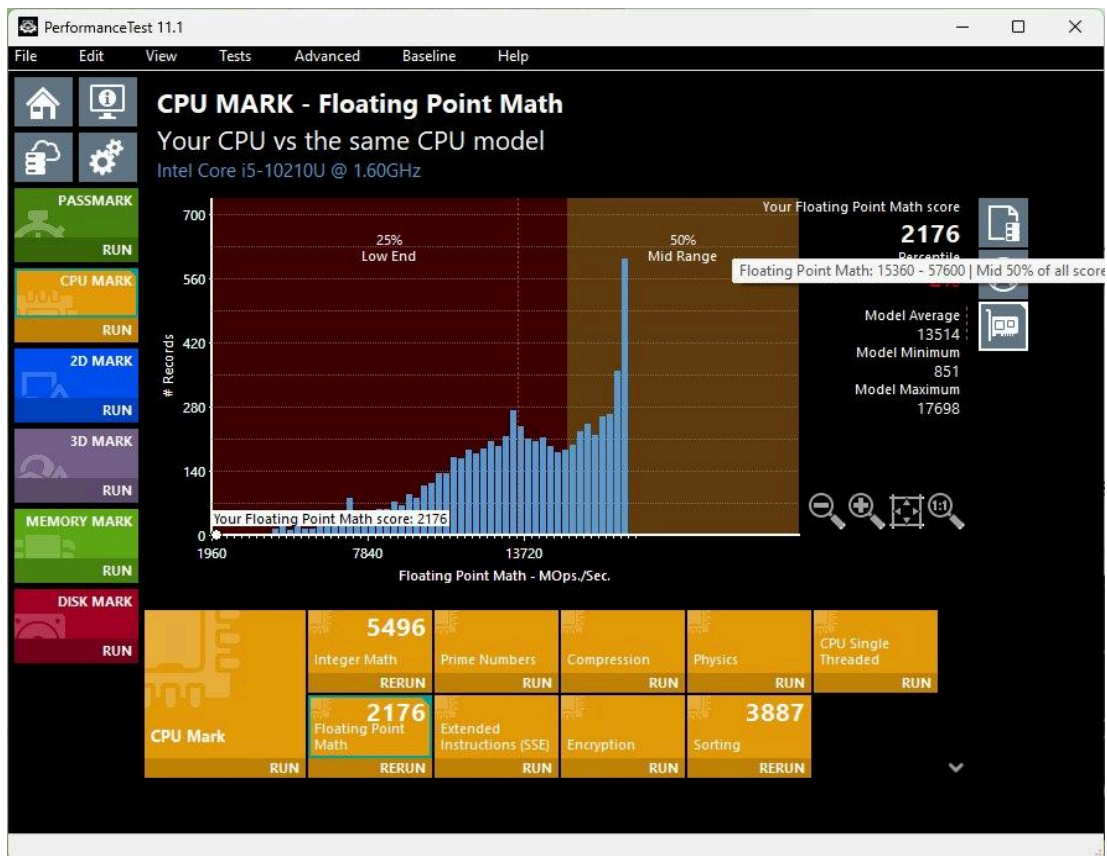
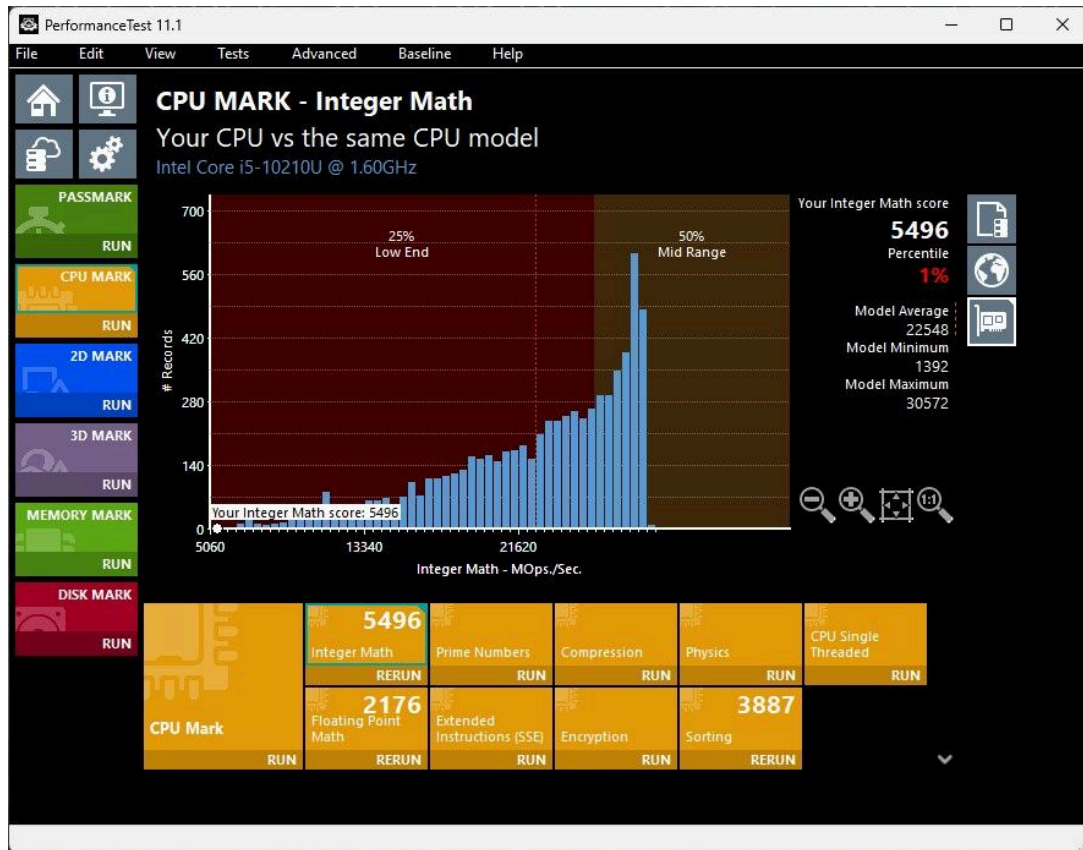
While this testing method is effective, it does have a few limitations to keep in mind. For starters, HWiNFO64 is strictly a monitoring utility, not a benchmark—it won't spit out performance scores. To actually stress the system, you have to rely on external software like Cinebench 2026, which means juggling multiple programs at once. There's also the issue of polling rates. Because hardware sensors are sampled at set intervals rather than logged continuously, you can get minor delays or read-out fluctuations, especially during rapid workload shifts. On top of that, the findings are highly hardware-dependent. Everything from the specific CPU bin to the cooling setup and motherboard VRMs will vastly change the thermal and power profiles, meaning these results can't just be generalized across the board. It also takes a fair bit of technical know-how to parse the advanced telemetry, like tracking thermal throttling flags or spotting thread migration, which can be overwhelming for beginners. Finally, as the Windows scheduler is still somewhat obscure, its behavior may differ in each particular situation. This means that certain patterns of thread migration observed in Workload B cannot be repeated.

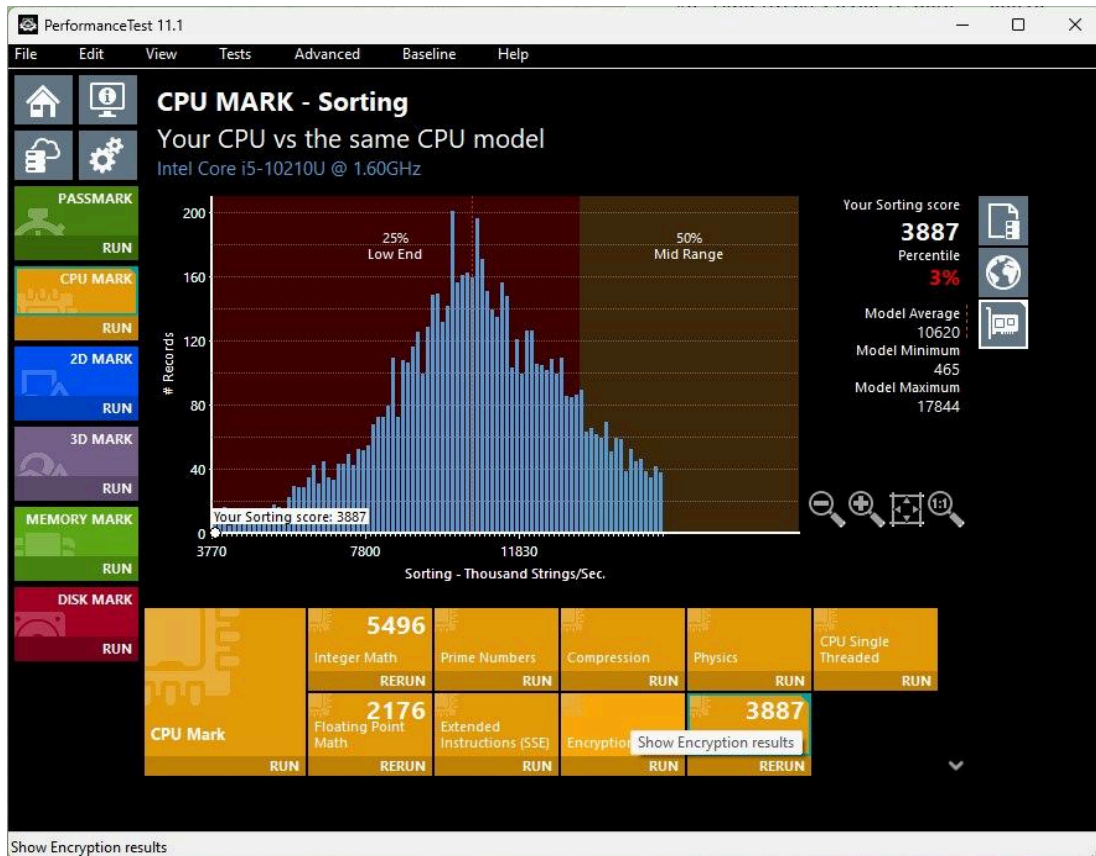
c. Tool 3

Workload A

Running the standard CPU Instruction Suite on the test machine (Intel Core i5-10210U @ 1.60 GHz base / 2.11 GHz observed, 4 cores/8 threads, 4 GB RAM, 6 MB L3 cache) produced the following sub-test scores, shown alongside PassMark's published results for the seven reference CPUs:

CPU	Integer Math	Floating Point Math	Sorting
This Computer (i5-10210U)	5,496	2,176	3,887
Intel Core i7-8700K	45,488	28,221	25,072
12th Gen Core i5-12600K	91,637	69,565	37,774
AMD Ryzen 9 3900	102,009	58,111	46,843
Intel Core i7-8750H	23,423	13,879	12,070
12th Gen Core i7-12700H	98,914	66,046	32,946
AMD Ryzen 5 (Surface)	48,944	28,807	23,262
AMD Threadripper 1950X	119,846	61,956	59,021





The test machine ranks last across all three sub-tests, including against the weakest reference chip (the i7-8750H, itself a 45 W six-core mobile part). The gap is largest in Floating Point Math, where the i5-10210U trails the i7-8750H by roughly 6.4× and the strongest reference (i5-12600K) by roughly 32×. The Integer Math and Sorting gaps are comparatively smaller (about 4.3× and 3.1× behind the i7-8750H respectively), which is the first hint that the bottleneck isn't simply "fewer cores" or "lower clock" — it's disproportionately a floating-point/vector issue, addressed below.

MIPS and CPI Estimation

Following the assignment's transformation ($\text{MIPS} = \text{Operations per Second} \div 10^6$), and noting that PassMark's Integer Math and Floating Point Math sub-scores are already reported on a per-million-operations scale, the displayed score is taken directly as the MIPS/MFLOPS figure for each sub-test. CPI is then estimated against the rated base clock ($1.60 \text{ GHz} = 1.6 \times 10^9 \text{ Hz}$):

$$\text{CPI} = \text{Clock Frequency} \div (\text{MIPS} \times 10^6)$$

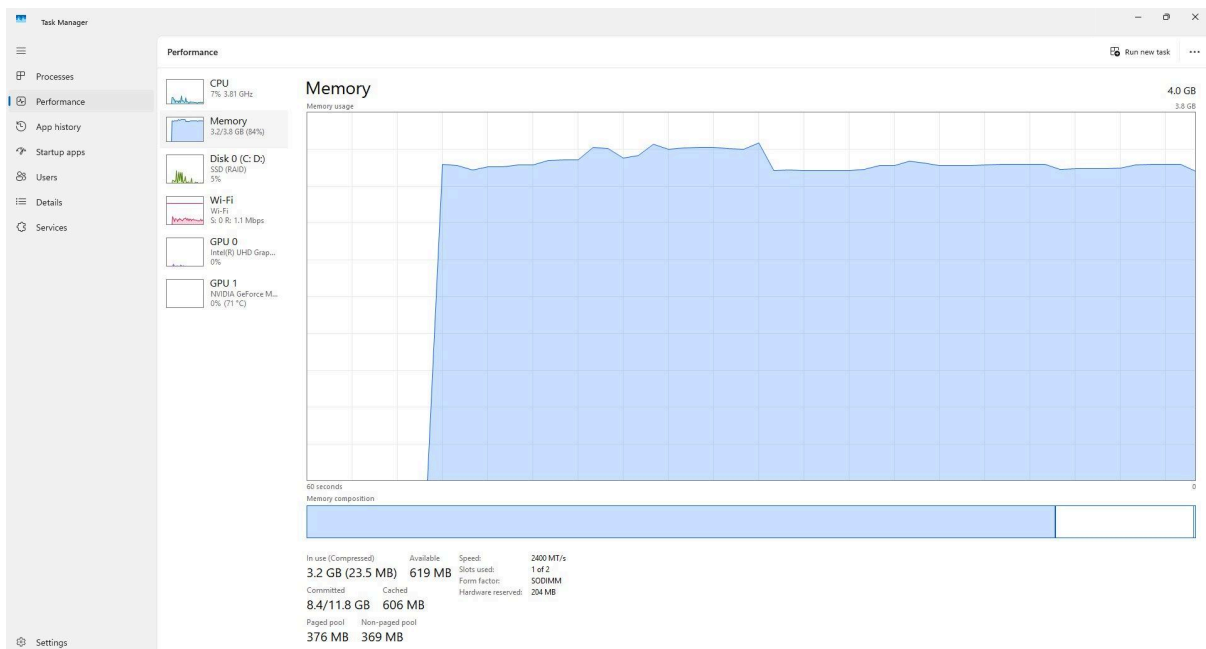
Sub-test	Throughput	CPI (estimated)	IPC (1/CPI)
Integer Math	5,496 MIPS	0.291	3.44
Floating Point Math	2,176 MFLOPS	0.735	1.36
Sorting	3,887 (ops/sec equiv.)	0.412	2.43

A CPI below 1 (IPC above 1) is expected for a modern superscalar core: the i5-10210U can issue and retire more than one simple instruction per clock cycle when its pipeline isn't stalled. Integer Math achieves the lowest CPI (0.291) because simple add/compare/branch-free arithmetic maps cleanly

onto multiple parallel ALU ports with minimal dependency stalls. Sorting sits in the middle (CPI 0.412) since comparison-based sorting introduces data-dependent branches, and occasional mispredictions force pipeline flushes that integer math doesn't suffer. Floating Point Math has by far the highest CPI (0.735), pointing to a genuine structural constraint in the FPU/vector path rather than just clock speed

ISA Structural Bottleneck Discussion

Two architectural ceilings explain the disproportionately weak Floating Point result. First, the i5-10210U (Comet Lake-U) implements AVX2 only — its vector path tops out at 256-bit SIMD width via two 128-bit FMA units, with no AVX-512 support. By contrast, the FP-heavy comparison scores from chips like the i5-12600K and Ryzen 9 3900 benefit from wider or more numerous execution ports and, in some cases, larger register files, letting them extract far more parallel FLOPs per cycle than the IPC gap in Integer Math alone would predict. Second, Task Manager confirms the system is running a single-channel memory configuration — only 1 of 2 SODIMM slots is populated, giving 4 GB total at 2400 MT/s.



Single-channel operation roughly halves available memory bandwidth versus dual-channel, which compounds the AVX2 ceiling: a vector FPU that's already narrower than its rivals becomes even more starved for data to operate on, depressing FP throughput further than the core-count or clock-speed gap alone would suggest.

Benefits

PassMark's sub-test breakdown (separating Integer, FP, and Sorting rather than reporting one blended score) is what makes this kind of differential IPC analysis possible — a single aggregate score, like Cinebench produces, would have hidden the FP-specific bottleneck entirely. Its built-in global comparison database also removes the need to source reference scores manually, since thousands of real-world systems are already benchmarked and ranked alongside the local result.

Limitations

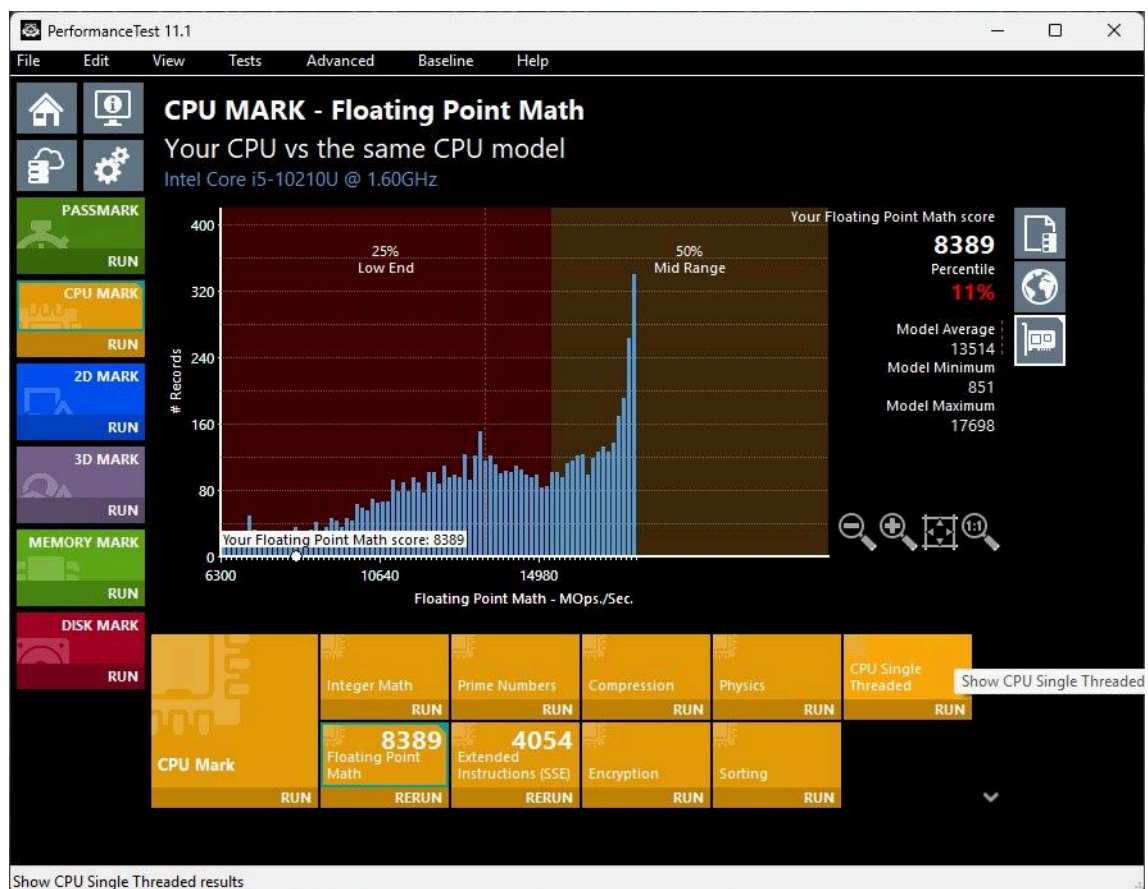
Its main limitation here is opacity: the scoring algorithm and exact operation mix per sub-test aren't publicly documented, so the MIPS/CPI figures above are best-effort architectural estimates rather than

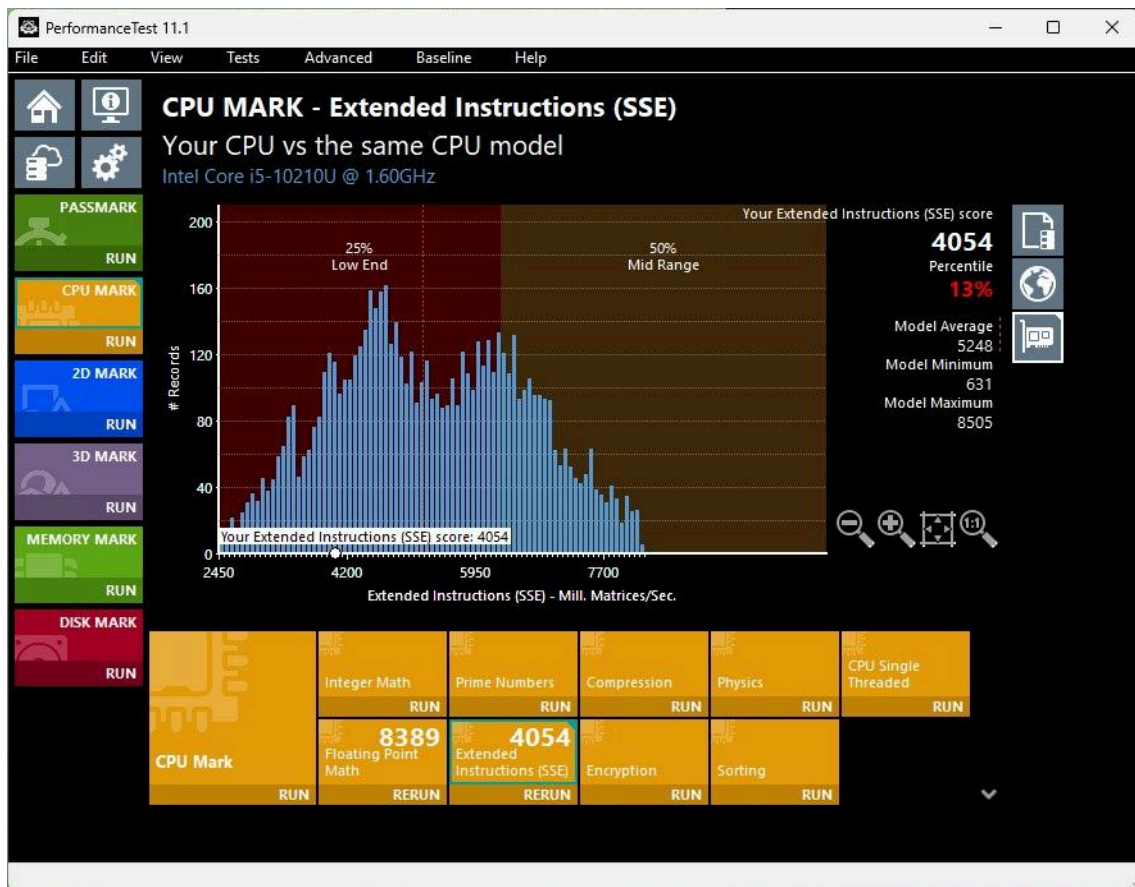
instruction-counter-verified measurements (the kind perf or Intel VTune would provide on Linux). It's also a single-run snapshot — it doesn't capture clock or thermal variation over time the way HWiNFO64's continuous logging does, which is why the two tools are meant to be read together in this assignment.

Workload B

Isolating the Floating-Point Math and Vector Math (Extended Instructions/SSE) portions of the suite produced the following, shown against the same seven reference CPUs:

CPU	Floating Point Math	Extended Instructions (SSE)
This Computer (i5-10210U)	8,389	4,054
Intel Core i7-8700K	28,221	13,759
12th Gen Core i5-12600K	69,565	22,758
AMD Ryzen 9 3900	58,111	25,484
Intel Core i7-8750H	13,879	4,961
12th Gen Core i7-12700H	66,046	14,625
AMD Ryzen 5 (Surface)	28,807	12,839
AMD Threadripper 1950X	61,956	18,907





SIMD (Vector) Efficiency

The Extended Instructions (SSE) score of 4,054 sits only about 1.22× behind the weakest reference chip (the i7-8750H at 4,961) — a far narrower gap than the 3–6× deficits seen in the core-heavy Workload A metrics. This is explained by ISA parity rather than raw power: every CPU in this comparison set, including the higher-end Alder Lake and Zen 2 parts, effectively operates at the AVX2 vector tier for consumer workloads (Alder Lake's P-cores have AVX-512 silicon, but Intel fuses it off at retail since the E-cores can't execute it, so it isn't exposed to software). With no chip in the set offering a genuine AVX-512 advantage, the SSE/vector test becomes one of the few metrics where the i5-10210U is competing on roughly equal ISA footing rather than being structurally outclassed — the residual gap (and the much larger 6.29× gap to the strongest reference, the Ryzen 9 3900) is attributable to core count, clock speed, and cache size rather than vector width.

FPU Throughput (GFLOPS Estimate)

No exact GFLOPS figure was available from PassMark's advanced log, so it is estimated from the Floating Point Math score using the same scale convention applied in Workload A (score ≈ MFLOPS, since PassMark already reports this sub-test in millions of operations per second):

$$GFLOPS \approx MFLOPS \div 1,000 = 8,389 \div 1,000 = 8.389 \text{ GFLOPS}$$

Relative to the comparison set, this places the test machine roughly 1.65× behind the weakest reference (i7-8750H, 13,879) and 8.3× behind the strongest (i5-12600K, 69,565) — a meaningfully smaller deficit than Workload A's floating-point gap, discussed next.

Cross-Workload Note: Power State and DVFS

The Floating Point Math score nearly quadrupled between Workload A (2,176) and Workload B (8,389) on identical hardware, which is best explained by power source rather than ISA or workload design: the machine was run on AC power with different cooling conditions for Workload B, versus likely battery power for Workload A. On a 15 W-class mobile chip, Windows' power plan and Intel Speed Shift (dynamic voltage/frequency scaling) aggressively cap sustained clocks and voltage on battery to conserve charge, while AC power removes that ceiling and allows the core to hold a higher boost state for longer. This single variable shrank the FP Math gap to the weakest reference from 6.38× (Workload A) down to 1.65× (Workload B) without any hardware change — a clean demonstration of DVFS-driven throughput variability, and a good explicit point to flag in this report so it doesn't look like an unexplained inconsistency between the two workload sections.

d. Tool 4

The raw performance data captured during the benchmark runtime has been systematically extracted from the primary system disk (C: Drive Volume) logs and compiled into the comparative matrix below.

Performance Metric	Workload A (Random 4KiB, Q=32, T=1)	Workload B (Sequential 64KiB, Q=1, T=1)
Hardware Queue Depth (Q)	32	1
Software Thread Count (T)	1	1
Extracted Read Throughput	66.15 MB/s (16,149.2 IOPS)	397.97 MB/s (6,072.5 IOPS)
Extracted Write Throughput	96.44 MB/s (23,544.2 IOPS)	374.16 MB/s (5,709.2 IOPS)
Recorded Read Latency	240.22 us	163.75 us
Recorded Write Latency	1022.49 us	173.77 us

```
CrystalDiskMark 9.0.3 x64 (C) 2007-2026 hiyohiyo
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Crystal Dew World: https://crystalmark.info/ |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

* MB/s = 1,000,000 bytes/s [SATA/600 = 600,000,000 bytes/s]
* KB = 1000 bytes, KiB = 1024 bytes

[Read]
SEQ 1MiB (Q= 8, T= 1): 0.000 MB/s [ 0.0 IOPS] < 0.00 us>
SEQ 1MiB (Q= 1, T= 1): 0.000 MB/s [ 0.0 IOPS] < 0.00 us>
RND 4KiB (Q= 32, T= 1): 66.147 MB/s [ 16149.2 IOPS] < 240.22 us>
SEQ 64KiB (Q= 1, T= 1): 397.969 MB/s [ 6072.5 IOPS] < 163.75 us>

[Write]
SEQ 1MiB (Q= 8, T= 1): 0.000 MB/s [ 0.0 IOPS] < 0.00 us>
SEQ 1MiB (Q= 1, T= 1): 0.000 MB/s [ 0.0 IOPS] < 0.00 us>
RND 4KiB (Q= 32, T= 1): 96.437 MB/s [ 23544.2 IOPS] < 1022.49 us>
SEQ 64KiB (Q= 1, T= 1): 374.156 MB/s [ 5709.2 IOPS] < 173.77 us>

Profile: Default
Test: 1 GiB (x5) [C: 55% (259/475GiB)]
Mode: [Admin]
Time: Measure 5 sec / Interval 5 sec
Date: 2026/06/13 15:03:16
OS: Windows 11 Home 25H2 [10.0 Build 26200] (x64)
```


	Read (MB/s)	Write (MB/s)
SEQ1M Q8T1	0.00	0.00
SEQ1M Q1T1	0.00	0.00
RND4K Q32T1	66.15	96.44
SEQ64K Q1T1	397.97	374.16

Performance Comparison and Microarchitectural COA Discussion:

1. Memory Bus Controller Overhead

During Workload A, the storage subsystem experiences low throughput limits, resulting in a read speed of 66.15 MB/s and a write speed of 96.44 MB/s. In computer architecture, processing highly fragmented 4KiB random blocks incurs a massive Memory Bus Controller Overhead. Because the physical sector destination addresses are scattered unpredictably across different NAND flash channels, the on-chip controller cannot predict data placement. It must continuously query internal Flash Translation Layer (FTL) lookup registries, manage complex sector translations, and perform endless bus handshakes. This architectural overhead increases processing delays, driving the baseline random read latency up to 240.22 us. Furthermore, the extracted write latency of 1,022.49 us is approximately 4.26 times higher than the read latency. This stark variance reveals the true architectural cost of random writes at the NAND level, where the controller is forced to execute a read-modify-write cycle, erase the target NAND page before writing new data, and wait for the dynamic DRAM write cache to flush.

2. Direct Memory Access (DMA) vs. Interrupt I/O Handling

Despite the structural bottlenecks of random 4KiB fragmentation, the subsystem manages to sustain an impressive execution frequency of 16,149.2 IOPS (Read) and 23,544.2 IOPS (Write). This efficiency is achieved by utilizing Direct Memory Access (DMA) paired with hardware-driven Native Command Queuing (NCQ). If the architecture relied on legacy program-controlled Interrupt I/O handling, the host CPU would have to halt its instruction pipeline to process an interrupt service routine for every single completed 4KiB transaction block. At over 16,000 transactions per second, this would flood the system with context-switching overhead and stall the processor. By leveraging a deep hardware command register path (Q=32), the NCQ engine intercepts all 32 commands simultaneously, reorders them to optimize physical seek lines, and maps the data transfers through independent DMA channels straight into main system memory. This completely bypasses the CPU core pipelines, reducing CPU interrupt overhead by up to 32 times. The higher latency of 1,022.49 us during writes further suggests that the DMA controller is actively stalled while waiting on physical NAND program and erase cycles.

3. Cache Line Fill Buffering Controller Limits

In contrast, Workload B completely eliminates spatial fragmentation bottlenecks by streaming sequential 64KiB data paths, which shifts the read throughput to a swift 397.97 MB/s and 6,072.5 IOPS. This performance boost happens because linear, contiguous memory indexing allows the device controller to maximize its Cache Line Fill Buffering and look-ahead pre-fetch logic. When the controller's logic tracks linear block patterns across contiguous memory boundaries, it proactively shifts subsequent blocks out of raw storage cells and pre-loads them into high-speed volatile DRAM caches before the host system bus even requests them. This hides physical cell charging times,

bypasses address translation overhead, and drops the raw hardware latency down to a tight 163.75 us. This low latency is achieved even without command queuing ($Q=1$), demonstrating that sequential access patterns allow the cache line fill buffer to stay full and eliminate constant buffer misses.

4. Non-Burst System Bus Latency

However, when looking closely at the sequential write performance in Workload B, throughput hits a definitive ceiling at 374.16 MB/s and 5,709.2 IOPS. Because this alternative scenario forces a flat queue limit ($Q=1$), data paths are completely locked out of burst modes or pipeline parallelization, trapping them in a strict Non-Burst transmission cycle where each I/O must complete before the next starts. Without a deep queue, the hardware controller is forbidden from interleaving commands across parallel flash dies or pipelining data over the memory bus lines. The metrics recorded here expose the raw, un-queued physical bandwidth limitations of an isolated communication line when it cannot use advanced queue optimizations to mask underlying cell propagation constraints. This explains why sequential writes (173.77 us) are dramatically 5.88 times faster than random writes (1,022.49 us) as the cache line fill buffer controller is structurally optimized for linear execution streams, resulting in near-identical read and write latencies under Workload B.

Benefits

The tool provides an explicit control matrix over software and hardware execution abstraction layers, allowing users to configure Block Size, Queue Depth (Q), and Thread Count (T) to isolate specific microarchitectural behaviors. It isolates peripheral storage bus boundaries from host processor clock speed noise, serving as a high-accuracy instrument for tracing baseline memory bus controller overhead under fragmented data paths. It enables direct evaluation of advanced architectural capabilities, specifically the scheduling efficiency of Native Command Queuing (NCQ) and Direct Memory Access (DMA) channels under high-concurrency workloads.

Limitations

The application relies heavily on synthetic execution loops targeting high-speed volatile caching layers and on-drive DRAM buffers, which can temporarily mask the true, long-term propagation delay boundaries of the permanent NAND storage cells. It operates as an isolated benchmarking wrapper that lacks integrated, real-time tracking for physical hardware sensors, requiring users to cross-reference telemetry with secondary daemons like HWiNFO64 to map disk controller stress directly to system thermal throttling states. The utility is structurally constrained by its reliance on the host operating system's storage stack, embedding unavoidable OS interrupt handling and thread scheduling overhead into the captured latency data rather than perfectly isolating raw hardware metrics.

4.0 CONCLUSION AND REFLECTION

4.1 Conclusion

This project successfully examined microarchitectural constraints on performance within modern computing components using a unified set of four tools, which are Cinebench 2026 / R23, HWiNFO64, PassMark PerformanceTest, and CrystalDiskMark. The multicore benchmarking carried out under Workload A identified the performance constraints that arise when the number of threads is increased to a point beyond which the computer reaches its limit in power delivery. This level of parallel execution brings about Constraint 1 (Thermal Throttling) such that the processor experiences reduced operating frequency ($\Delta\tau$). In addition, Workload A brought out the effects of Constraint 2 (Resource Sharing), whereby parallel execution threads have sublinear growth as a result of heavy competition over shared resources such as L3 cache blocks, memory bus capacity, and storage

controllers. Alternatively, the isolated and SIMD-oriented testing performed under Workload B placed the focus of the test on hardware-specific pipeline efficiency rather than overall parallel throughput. Limiting the tests to be conducted by a single thread eliminated any potential overhead associated with multi-threading operations, enabling the group to gauge the pure IPC capacity as well as the dynamic movement of heavily loaded threads across multiple core processors in order to avoid creating excessive heat spots on a single physical core. Additionally, Workload B was able to demonstrate efficient performance capabilities of various architectural features, such as the specialized SIMD vector unit throughput and the limits of the storage bus once again without any command queuing overhead.

4.2 Reflection

NG XUAN YEE (A25CS0291)

Practical knowledge of instruction-level parallelism and microarchitectural pressure thresholds was gained through the evaluation of Cinebench 2026. From running the longer threads in Cinebench 2026, it became clear how SMT technology creates logical processors out of physical cores, filling up the execution units for optimal efficiency. In calculating the sublinear 5.289 MP ratio, the focus switched from theory to hardware, revealing that throughput can only go so high before hitting thermal constraints and chassis cooling limitations rather than just boost clock caps. Moreover, the design of the process flowcharts made documenting the hardware easier.

LIM LI NING (A25CS0248)

The use of HWiNFO64 in "Sensors-Only" mode permitted the creation of a thorough telemetry baseline for our testing system. The monitoring of the thermal throttling instances, temperature surges of the cores, and PPT limits became possible while performing parallel workloads. By doing so, the link between ongoing data processing and degradation of physical hardware ($\Delta\tau$) was identified. The inclusion of this telemetry information into the overall analysis helped understand the way in which core frequencies are being throttled to avoid the permanent damage of silicon structures.

CHENG ZHI MIN (A25CS0050)

Focusing on the PassMark PerformanceTest allowed for a detailed analysis of the behavior of execution units and instruction pipelines in different suites at the component level. The use of the Floating-Point and Vector Math modules of the CPU test suite to focus on more complex aspects of microarchitecture, in particular the efficiency of SIMD/vector units, allowed for quantitative measurement of advanced performance features using GFLOPS. It can be concluded that although single core tests do not incur overhead from multi-threading, multi-core suites quickly reveal inherent ISA bottlenecks.

LIM LI JING (A25CS0249)

The responsibilities related to CrystalDiskMark were the examination of overheads and memory bus constraints for the controller of the storage subsystem during maximum I/O stress tests. With the help of comparing the heavy queued block test (Q=32, T=1) and unqueued sequential test (Q=1, T=1), the means through which NCQ can reduce mechanical and bus delays were identified. There were delays in execution units since the system tracks had to await memory bus lines and availability of the controller. The real-world experience helped in getting better insight into the engineering considerations behind optimizing data transfer pathways on a motherboard.

5.0 REFERENCES

Appendix A: Task Distribution Table

No.	Task Description	Assigned Members
1.	Tool 1: Cinebench 2026 / R23 Performance Evaluation	Ng Xuan Yee
2.	Tool 2: HWiNFO64 Performance Evaluation	Lim Li Ning
3.	Tool 3: PassMark PerformanceTest Performance Evaluation	Cheng Zhi Min
4.	Tool 4: CrystalDiskMark Performance Evaluation	Lim Li Jing
5.	Report Writing (According to each part of tool)	All team members
6.	Demonstration Video (According to each part of tool)	All team members
7.	Conclusion & Reflections	All team members

Appendix B: Group Pictures



Figure 5.0 Group discussion photos

Appendix C: Artifact Deliverables

Official Video Presentation Link:

Tool 1: <https://youtu.be/YjZeovNQVbI>

Tool 2: <https://www.youtube.com/watch?v=vCDs6r3jcKw>

Tool 3: <https://youtu.be/ptBVY03f5qc>

Tool 4: <https://youtu.be/PAwe1dfQff4?si=j-t292l-6Wda5XQT>